ENTREGA #1

AUTORES:

DANIEL FELIPE SÁNCHEZ PERAFAN
JUAN MANUEL RIVERA
NESTOR DAVID TORRES ROJAS

Presentado a:
JESUS EMILIO PINTO LOPERA
Docente

UNIVERSIDAD DE LA AMAZONIA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
FLORENCIA, CAQUETÁ
2026-1

TABLA DE CONTENIDO

INTRODUCCIÓN 4
METODOLOGÍA 5
Regresión 5
Dataset de regresión: biometría de e-sports 5
Correlación de cada variable numérica con `Reaction_Time_ms` 7
Capacidad individual de cada atributo como nodo raíz 8
Hold-Out y preprocesamiento dentro de Cross-Validation 11
Escalado según el algoritmo utilizado 12
Clasificación 13
Dataset de clasificación: reservas hoteleras 13
Carga, separación y preprocesamiento de los datos 15
Evaluación individual de atributos como nodo raíz 17
Herramientas utilizadas 18
Métricas de evaluación 19
RESULTADOS 21
Regresión 21
Resultados del Árbol de Decisión Regressor 21
Análisis de subajuste, ajuste óptimo y sobreajuste 22
Interpretación del árbol final y nodo raíz 22
Resultados del modelo Random Forest Regressor 24
Resultados del modelo KNN Regressor 27
Selección del número de vecinos 28
Visualización del vecindario y efecto del tamaño de entrenamiento 29
Comparación final de los modelos 30
Clasificación 35
Clasificación con Árbol de Decisión 35
Ajuste de los umbrales de clasificación 36

Clasificación con Random Forest 38

Clasificación con KNN 43

Comparación final de modelos 46

Análisis no supervisado mediante K-Means 47

CONCLUSIÓN 53

INTRODUCCIÓN

Este proyecto estudia dos problemas supervisados mediante técnicas de aprendizaje automático. El primer problema es de regresión y busca estimar el tiempo de reacción de un jugador de e-sports a partir de variables biométricas y de comportamiento. El segundo problema es de clasificación binaria y busca determinar si una reserva hotelera será cancelada.

En ambos escenarios, la dificultad principal no se encuentra únicamente en entrenar un modelo predictivo, sino en determinar si los patrones encontrados pueden generalizarse hacia nuevos registros. Por esta razón, el análisis incorpora procesos de selección de variables, control de complejidad y validación cruzada, buscando comparar no solo el rendimiento obtenido, sino también la estabilidad de cada algoritmo frente a datos no utilizados durante el entrenamiento.

En ambos casos se comparan tres familias de modelos: Árbol de Decisión, Random Forest y KNN. El propósito no es solamente obtener una predicción, sino construir un experimento reproducible que permita analizar la selección de variables, el efecto de la regularización, la estabilidad de los modelos y la diferencia entre el desempeño durante la validación y el desempeño sobre datos no vistos.

La metodología separa los datos en entrenamiento y prueba mediante un esquema Hold-Out 70/30. El conjunto TEST permanece aislado durante la selección de hiperparámetros. Todas las decisiones de ajuste se realizan dentro de TRAIN usando Cross-Validation de 5 folds. De esta manera se reduce el riesgo de sobreajuste y de data leakage, y la evaluación final representa mejor el comportamiento esperado del modelo ante nuevos registros.

Adicionalmente, se realiza un análisis no supervisado mediante el algoritmo K-Means con el objetivo de identificar posibles agrupamientos dentro de los datos de reservas hoteleras y comparar dichos grupos con la distribución real de la variable objetivo.

METODOLOGÍA

Para el desarrollo del proyecto se siguió un flujo experimental basado en aprendizaje automático supervisado y no supervisado. Primero se realizó la preparación y análisis exploratorio de los conjuntos de datos, seguido de la separación entre entrenamiento y prueba, el preprocesamiento de variables y la evaluación de diferentes algoritmos mediante métricas específicas. En los modelos supervisados se utilizaron estrategias de validación cruzada para comparar configuraciones y seleccionar los modelos con mejor desempeño.

Regresión

Dataset de regresión: biometría de e-sports

El trabajo comienza con la lectura y revisión del archivo `data/esports_biometrics.csv`. Este conjunto contiene 250.000 registros y 13 variables relacionadas con la biometría y el desempeño de jugadores de E-Sports. Cada registro representa una observación de un jugador y contiene información sobre edad, hábitos de juego, condiciones de la pantalla, actividad durante la partida y variables fisiológicas.

La variable que se desea predecir es `Reaction_Time_ms`, que corresponde al tiempo de reacción del jugador medido en milisegundos. Por tanto, el problema se plantea como una tarea de regresión, porque la salida es un valor numérico continuo.

Antes de entrenar los modelos se excluyen `Player_ID`, debido a que es un identificador único sin significado predictivo, y `Match_Outcome`, porque representa el resultado de la partida y podría contener información disponible después del desempeño que se pretende predecir. Esta decisión evita que el modelo utilice información que no estaría disponible en un escenario de predicción real.

Como se muestra en la Figura 1, se utilizan estas variables predictivas:

N.º	Player_ID	Age	Game_Genre	PlayTime_Hours	Sleep_Hours	Energy_Drinks	Screen_Brightness	Blue_Light_Filter	APM	Heart_Rate_BPM	HRV_Score	Reaction_Time_ms	Match_Outcome
0	PL_000001	24	FPS	1.41	8.88	0	43.4	Yes	107	78	95.02	268.96	Loss
1	PL_000002	22	FPS	4.43	7.45	2	88.0	Yes	110	102	82.92	281.90	Win
2	PL_000003	25	Battle Royale	3.75	8.10	1	24.8	Yes	152	96	89.13	288.57	Win
3	PL_000004	29	FPS	1.76	6.69	0	80.2	No	151	65	80.41	279.76	Win
4	PL_000005	22	FPS	5.79	8.17	1	34.4	No	148	95	95.71	229.62	Loss

Figura 1. Tipos de datos del dataset regresion.

En el bloque de código de la figura 2 se muestra la importación de las librerías, la definición de la carpeta de resultados y la lectura del archivo `esports_biometrics.csv` utilizado en todo el trabajo.

```python
from pathlib import Path
import time
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import KFold, cross_validate, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor

RESULTS_DIR = Path('resultados_regresion')
RESULTS_DIR.mkdir(exist_ok=True)

DATA_PATH = Path('data/esports_biometrics.csv')
df = pd.read_csv(DATA_PATH)

print('Dimensión:', df.shape)
display(df.head())
print(df.dtypes)
print(df.isna().sum())
```

Figura 2. Importación de librerías, lectura y revisión inicial del dataset.

Después de cargar el archivo, se revisan los estadísticos descriptivos, los tipos de datos y las relaciones lineales entre las variables numéricas. Esta revisión permite conocer la escala de los atributos, detectar valores nulos y obtener una primera aproximación de cuáles variables podrían estar relacionadas con el tiempo de reacción.

En el bloque de código de la figura 3 se muestra la obtención de los estadísticos descriptivos y el cálculo de la correlación de Pearson con la variable objetivo.

```python
display(df.describe(include='all').T)

num_cols_all = df.select_dtypes(include='number').columns.tolist()
corr = (
    df[num_cols_all]
    .corr(numeric_only=True)['Reaction_Time_ms']
    .drop('Reaction_Time_ms')
    .sort_values()
)
```

Figura 3. Estadísticos descriptivos y cálculo de correlaciones.

Correlación de cada variable numérica con `Reaction_Time_ms`

Para realizar una primera exploración de los datos se calculó la correlación de Pearson entre cada variable numérica y `Reaction_Time_ms`. Este coeficiente toma valores entre −1 y 1 y mide la intensidad y dirección de una relación lineal: un valor positivo indica que ambas variables tienden a aumentar juntas, mientras que un valor negativo indica que, cuando una aumenta, la otra tiende a disminuir. Cuanto más cercano esté el valor absoluto a 1, más fuerte es la relación lineal; un valor cercano a 0 indica poca o ninguna relación lineal.

Esta medición se utiliza únicamente como análisis exploratorio. Una correlación alta no demuestra que una variable cause cambios en el tiempo de reacción y una correlación baja no significa necesariamente que la variable sea inútil para los modelos, porque puede participar en relaciones no lineales o en interacciones con otros atributos.

En la figura 4 se muestra la generación de las correlaciones entre las variables numéricas y el tiempo de reacción.

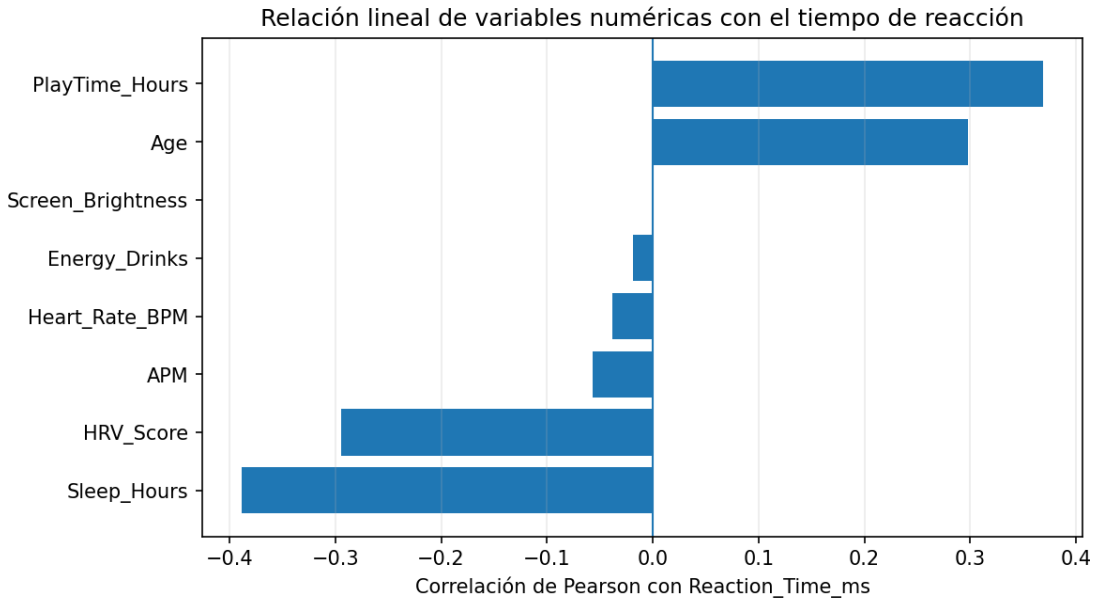


Figura 4. Correlaciones de Pearson del conjunto de datos.

Capacidad individual de cada atributo como nodo raíz

Se entrenaron modelos de profundidad 1 (decision stumps) para analizar la capacidad predictiva individual de cada variable y determinar cuál sería el nodo raíz más adecuado en un árbol de decisión. El desempeño se midió usando RMSE y R^2 en la evaluación, que se efectuó a través de una validación cruzada de 5 folds sobre el conjunto TRAIN.

N.º	Atributo	RMSE_CV_mean	RMSE_CV_std	MAE_CV_mean	R2_CV_mean	R2_CV_std	Ranking_RMSE
0	Sleep_Hours	18.1780	0.0418	14.4809	0.0970	0.0025	1
1	PlayTime_Hours	18.2157	0.0514	14.5211	0.0932	0.0032	2
2	Age	18.5596	0.0371	14.7865	0.0587	0.0020	3
3	HRV_Score	18.5954	0.0214	14.8290	0.0550	0.0028	4
4	APM	19.0811	0.0303	15.2027	0.0050	0.0009	5
5	Energy_Drinks	19.1061	0.0315	15.2207	0.0024	0.0005	6

6	Heart_Rate_BPM	19.1198	0.0275	15.2323	0.0010	0.0004	7
7	Blue_Light_Filter	19.1297	0.0274	15.2391	-0.0000	0.0001	8
8	Game_Genre	19.1297	0.0274	15.2392	-0.0001	0.0001	9
9	Screen_Brightness	19.1300	0.0268	15.2394	-0.0001	0.0001	10

Tabla 1. Desempeño individual de cada atributo como candidato a nodo raíz

Los hallazgos revelan que `Sleep_Hours` se sitúa en el primer puesto, con el RMSE medio más bajo (18.178) y el R^2 medio más alto (0.097). Esto señala que, entre los atributos analizados de manera individual, las horas de sueño son las que brindan más datos para calcular `Reaction_Time_ms` a través de una sola división del árbol.

`PlayTime_Hours` es el segundo en la lista y tiene un rendimiento muy similar al de `Sleep_Hours`. Aunque en menor medida, `Age` y `HRV_Score` también reflejan una notable capacidad individual. Por otro lado, `APM`, `Game_Genre`, `Screen_Brightness`, `Heart_Rate_BPM`, `Energy_Drinks` y `Blue_Light_Filter` tienen valores de R^2 que están cerca de cero y RMSE que son similares. Esto quiere decir que, si se usan en un árbol de profundidad 1 de manera aislada, tienen una capacidad limitada para explicar el tiempo de reacción.

En el bloque de código de la figura 5 se muestra el entrenamiento de un árbol tipo *stump* para cada atributo y la evaluación mediante `cross_validate`.

```python
for col in X_train.columns:
    pipe = Pipeline([
        ('prep', ColumnTransformer([
            ('feature', prep, [col])
        ])),
        ('model', DecisionTreeRegressor(
            max_depth=1,
            random_state=RANDOM_STATE
        )),
    ])

    scores = cross_validate(
        pipe,
        X_train[[col]],
        y_train,
        cv=cv5,
        scoring=scoring,
        n_jobs=1
    )
```

Figura 5. Evaluación de cada atributo como candidato a nodo raíz.

Este experimento (un árbol de profundidad 1 que utiliza un solo atributo a la vez) demuestra de manera cuantitativa que `Sleep_Hours` es el mejor candidato para ser nodo raíz y que `PlayTime_Hours` lo sigue muy de cerca. Los últimos seis atributos son casi indistinguibles de la casualidad ($R^2 = 0$).

En la figura 6 se muestra la comparación mediante validación cruzada de cada atributo como posible raíz de un árbol tipo stump.

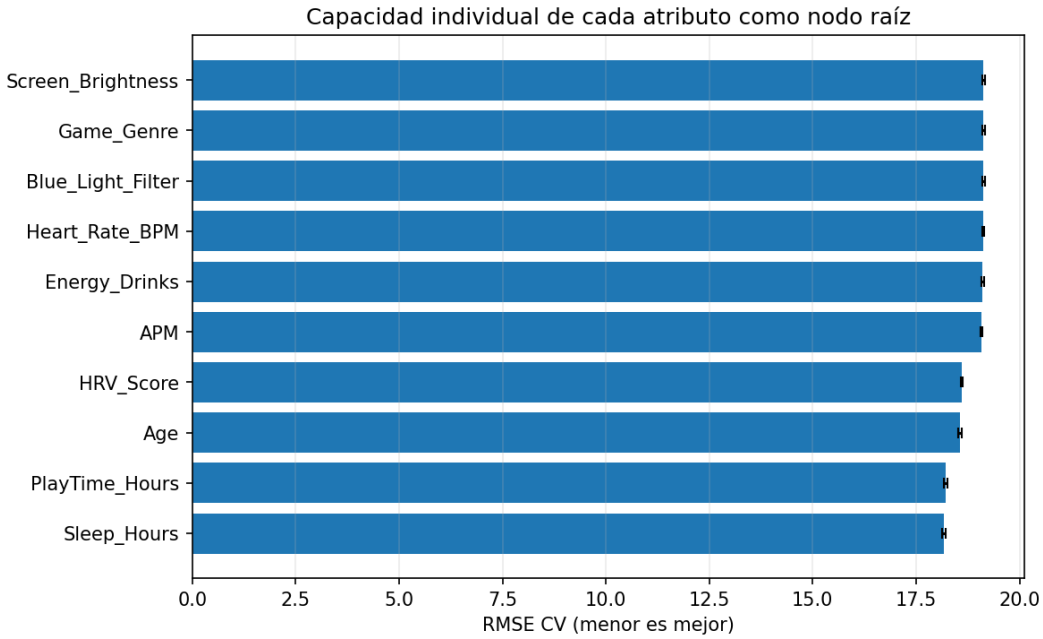


Figura 6. RMSE de validación cruzada por atributo.

Hold-Out y preprocesamiento dentro de Cross-Validation

Orden anti-fuga de datos

1. Hold-Out 70/30 (antes de cualquier imputación, codificación o escalado)
2. En cada fold de Cross-Validation, la imputación/codificación/escalado se ajusta (`fit`) exclusivamente a la porción de entrenamiento correspondiente a dicho fold, lo que previene que la validación cruzada "vea" estadísticos generados con datos que no están relacionados con el fold.
3. Una vez establecidos los hiperparámetros, el modelo final se reentrena para ajustar el preprocesamiento utilizando el TRAIN completo (175,000 filas) y se evalúa únicamente en TEST (75,000 filas).

En el bloque de código de la figura 7 se muestra la definición de la variable objetivo, el descarte de columnas y la división Hold-Out 70/30.

```python
target = 'Reaction_Time_ms'
drop_cols = ['Player_ID', 'Match_Outcome']

X = df.drop(columns=drop_cols + [target]).copy()
y = df[target].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)
```

Figura 7. Separación de TRAIN y TEST antes del preprocesamiento.

Escalado según el algoritmo utilizado

- KNN: Es obligatorio aplicar el `StandardScaler` a las variables numéricas (algoritmo fundamentado en la distancia euclidiana; sin escalar, `APM` en el rango de cientos prevalecería sobre `Sleep_Hours` en el rango de 0–11).

- Árbol de Decisión y Random Forest: la escala no se realiza. Los segmentos de un árbol se fundamentan en umbrales ordenados, no en magnitudes ni distancias, por lo que el escalado no tiene relevancia matemática para estos dos modelos. Esta es una elección más estricta que implementar un único pipeline "talla única" para los tres algoritmos respectivamente.

En el bloque de código de la figura 8 se muestra el pipeline específico de KNN, que imputa, escala las variables numéricas y codifica las variables categóricas.

```python
def make_knn_preprocessor():
    return ColumnTransformer(
        transformers=[
            ('screen', Pipeline([
                ('imputer', SimpleImputer(strategy='mean')),
                ('scaler', StandardScaler()),
            ]), ['Screen_Brightness']),
            ('hrv', Pipeline([
                ('imputer', SimpleImputer(strategy='median')),
                ('scaler', StandardScaler()),
            ]), ['HRV_Score']),
            ('num', StandardScaler(), other_numeric),
            ('cat', OneHotEncoder(
                handle_unknown='ignore',
                sparse_output=False,
                drop='if_binary',
            ), categorical_cols),
        ],
        verbose_feature_names_out=False,
    )
```

Figura 8. Preprocesamiento reproducible para KNN.

Clasificación

Dataset de clasificación: reservas hoteleras

El segundo problema abordado corresponde a una tarea de clasificación binaria utilizando información relacionada con reservas hoteleras. El objetivo consiste en predecir si una reserva será cancelada o no, mediante la variable objetivo is_canceled. A diferencia del problema de regresión, donde la salida corresponde a un valor continuo asociado al tiempo de reacción de jugadores de e-sports, en este caso el modelo debe asignar cada registro a una de dos categorías posibles: reserva cancelada o reserva no cancelada.

El conjunto de datos contiene variables relacionadas con las características de la reserva, comportamiento histórico del cliente y condiciones asociadas al proceso de compra, entre ellas tiempo anticipado de reserva (lead_time), país de procedencia del cliente, tipo de depósito, segmento del mercado, solicitudes especiales, cambios realizados en la reserva y variables relacionadas con estancias anteriores. Como se muestra en la Figura 9, se utilizan estas variables:

```
hotel                              str
is_canceled                      int64
lead_time                        int64
arrival_date_year                int64
arrival_date_month                 str
arrival_date_week_number         int64
arrival_date_day_of_month        int64
stays_in_weekend_nights          int64
stays_in_week_nights             int64
adults                           int64
children                       float64
babies                           int64
meal                               str
country                            str
market_segment                     str
distribution_channel               str
is_repeated_guest                int64
previous_cancellations           int64
previous_bookings_not_canceled   int64
reserved_room_type                 str
assigned_room_type                 str
booking_changes                  int64
deposit_type                       str
agent                          float64
...
reservation_status                 str
reservation_status_date            str
city                               str
```

Figura 9. Tipos de datos dataset clasificación.

Antes del entrenamiento de los modelos se realizó una revisión de la información disponible con el objetivo de identificar variables inconsistentes, valores extremos y posibles fuentes de fuga de información. Al igual que en el problema de regresión, las decisiones de preparación de datos se realizaron antes del ajuste definitivo de los modelos para garantizar que la evaluación representara un escenario real de predicción sobre datos no observados.

La variable objetivo presenta una distribución desbalanceada, donde la clase correspondiente a reservas no canceladas tiene una mayor representación dentro del conjunto de datos. Por esta razón, además de la exactitud global, se utilizaron métricas complementarias como Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC y PR-AUC, debido a que una

métrica como Accuracy puede presentar una interpretación limitada cuando existe diferencia entre la cantidad de observaciones de cada clase.

Para comprender la distribución de las clases antes del entrenamiento, se realizó un análisis inicial de la variable objetivo. Como se observa en la Figura 10, existe una diferencia en la cantidad de registros pertenecientes a cada categoría, situación que justifica el uso de métricas adicionales a la exactitud global.

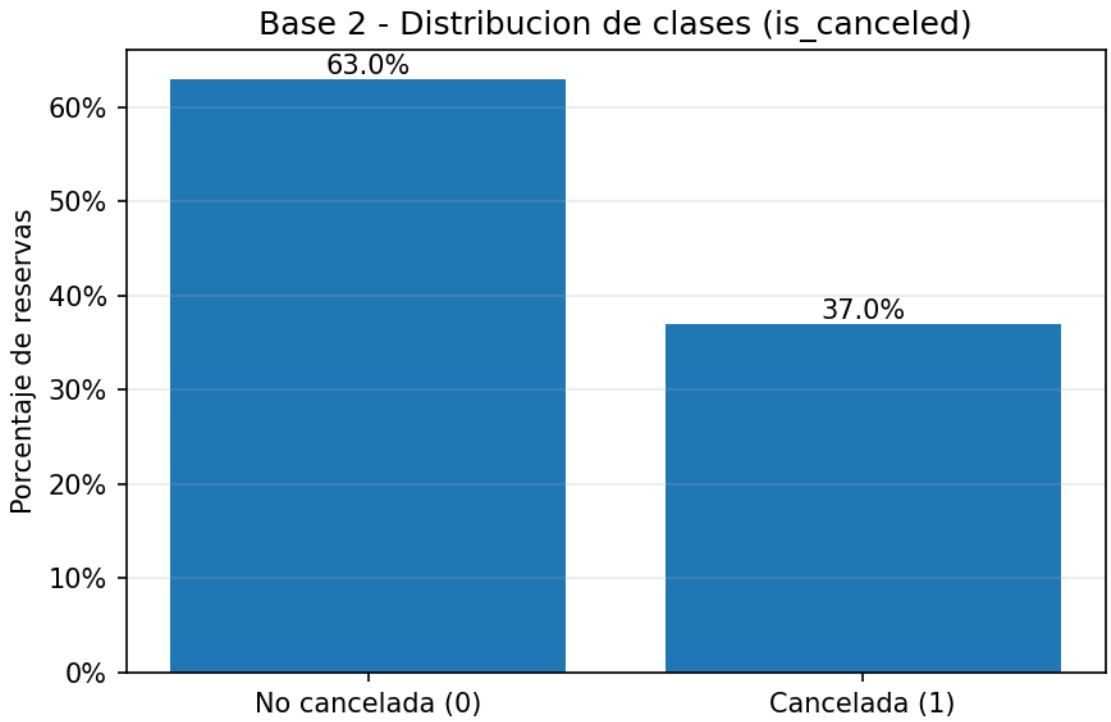


Figura 10. Distribución de las clases de la variable objetivo is_canceled

Carga, separación y preprocesamiento de los datos

Siguiendo la misma estrategia aplicada en regresión, los datos fueron divididos mediante un esquema Hold-Out 70/30 antes de realizar cualquier transformación. De esta manera, el conjunto TEST permanece separado durante la selección de hiperparámetros y solamente se utiliza en la evaluación final.

El preprocesamiento fue incorporado dentro de Pipeline para evitar que información del conjunto de prueba o de los folds de validación influya en el entrenamiento. Esta decisión

permite que operaciones como imputación, codificación de variables categóricas y escalamiento se ajusten únicamente con los datos disponibles en cada partición de entrenamiento.

Para los modelos basados en árboles no fue necesario aplicar escalamiento, debido a que estos algoritmos realizan divisiones mediante umbrales y no dependen de la distancia entre observaciones. En cambio, para KNN se aplicó normalización mediante StandardScaler, ya que su funcionamiento depende directamente del cálculo de distancias entre registros.

La validación cruzada utilizada fue de 5 folds, permitiendo comparar diferentes configuraciones de cada modelo y seleccionar aquellas con mejor equilibrio entre capacidad predictiva y estabilidad.

Como parte del análisis metodológico se compararon diferentes proporciones de separación TRAIN/TEST. Como se observa en la Figura 11, las configuraciones 60/40, 70/30 y 80/20 presentan comportamientos similares en las métricas evaluadas. Se mantuvo la partición 70/30, ya definida para el experimento, porque proporciona un equilibrio entre la cantidad de datos disponible para el entrenamiento y un conjunto TEST suficientemente amplio para realizar la evaluación final.

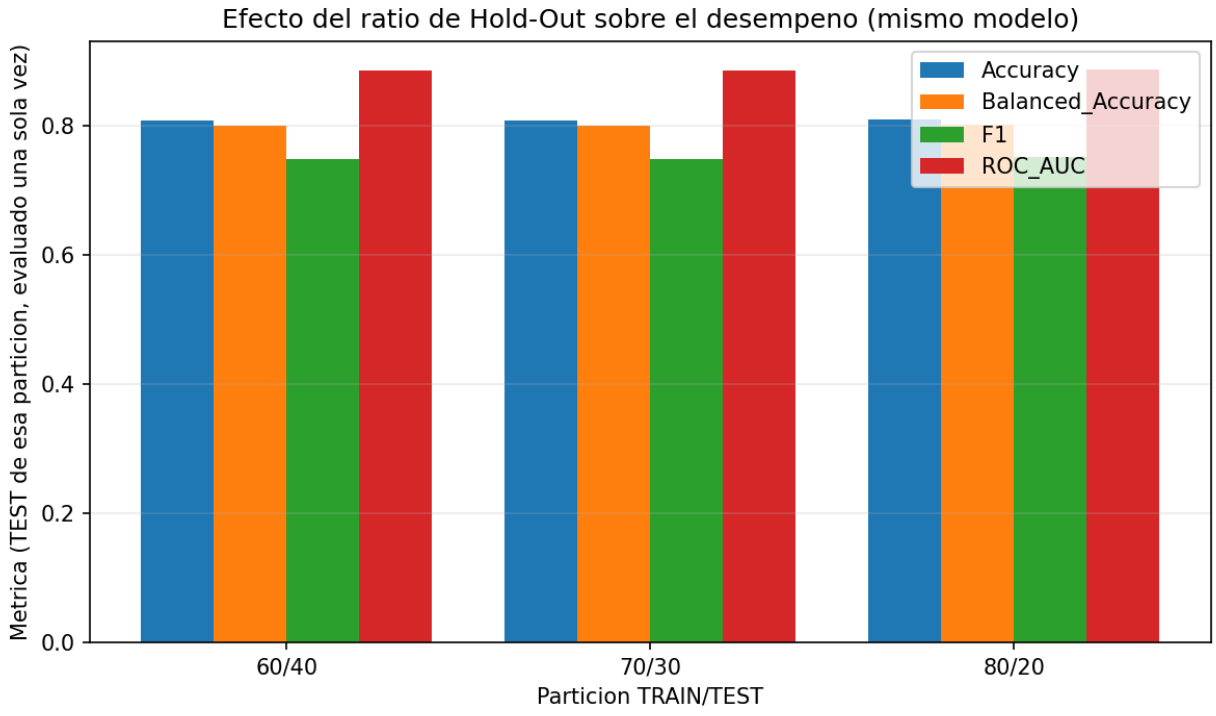


Figura 11. Evaluación de proporciones Hold-Out para entrenamiento y prueba.

```
# ------------------------------------------------------------------
# 3. Hold-Out 70/30 ESTRATIFICADO -- la particion que efectivamente
#    usa el resto del notebook (antes de cualquier imputacion,
#    agrupacion de categorias o escalado)
# ------------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=RANDOM_STATE, stratify=y
)
print(f"\nTRAIN: {len(X_train):,} ({len(X_train)/len(X):.1%})")
print(f"TEST : {len(X_test):,} ({len(X_test)/len(X):.1%})")
print("\nDistribucion TRAIN:")
print((y_train.value_counts(normalize=True).sort_index() * 100).round(2))
print("Distribucion TEST:")
print((y_test.value_counts(normalize=True).sort_index() * 100).round(2))
```

Figura 12. Separación del conjunto TRAIN y TEST antes del preprocesamiento.

Evaluación individual de atributos como nodo raíz

Antes de entrenar los modelos completos se evaluó la capacidad individual de cada atributo mediante árboles de profundidad limitada. Este procedimiento permite identificar qué variables tienen mayor capacidad inicial para separar las clases y analizar cuáles podrían generar las primeras divisiones dentro de un árbol de decisión.

Para identificar qué variables contienen mayor información inicial para separar las clases, se evaluaron árboles de profundidad limitada utilizando cada atributo de manera individual. La Figura 13 muestra la comparación obtenida mediante validación cruzada.

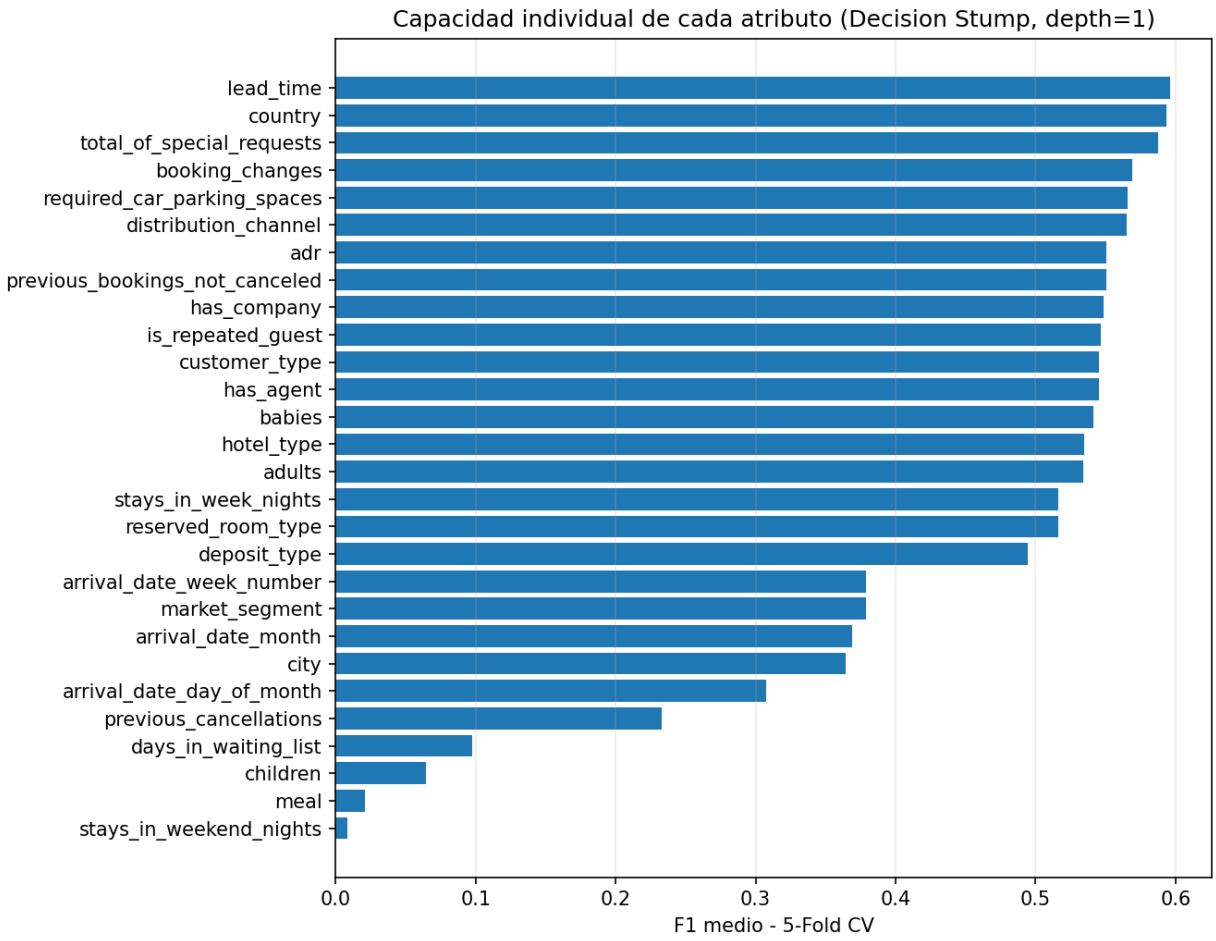


Figura 13. Capacidad individual de los atributos como nodo raíz en clasificación.

Los resultados muestran que lead_time presentó el mejor comportamiento individual, alcanzando un F1 medio de validación cruzada de 0.596 y una Balanced Accuracy de 0.619. Otras variables con comportamiento relevante fueron country y total_of_special_requests, que también mostraron capacidad para diferenciar entre reservas canceladas y no canceladas.

La importancia de estas variables no implica que expliquen por sí solas todo el comportamiento del fenómeno. Una reserva puede cancelarse por múltiples factores combinados, por lo que el análisis individual únicamente permite identificar atributos con información inicial útil para el modelo.

Herramientas utilizadas

El proyecto se desarrolló en Python mediante Jupyter Notebook. Se utilizó Pandas para cargar, explorar y transformar los datos; NumPy para las operaciones numéricas; y Matplotlib

para elaborar las gráficas. Scikit-learn se empleó como biblioteca principal para el preprocesamiento, el entrenamiento y la evaluación de los modelos.

La separación entre entrenamiento y prueba se realizó con train_test_split. Para la validación cruzada de cinco particiones se utilizaron KFold en regresión y StratifiedKFold en clasificación, este último para conservar aproximadamente la proporción de las clases en cada partición. Las funciones cross_validate y GridSearchCV permitieron evaluar los modelos y seleccionar sus hiperparámetros. Además, cross_val_predict se utilizó para obtener probabilidades fuera de muestra dentro de TRAIN y ajustar los umbrales de clasificación.

El preprocesamiento se organizó mediante Pipeline y ColumnTransformer, de modo que las transformaciones se ajustaran con los datos de entrenamiento correspondientes a cada partición. Se emplearon SimpleImputer para los valores faltantes, OneHotEncoder para las variables categóricas y StandardScaler para el escalado numérico en los procedimientos basados en distancias. Los modelos supervisados utilizados fueron DecisionTreeRegressor y DecisionTreeClassifier, RandomForestRegressor y RandomForestClassifier, y KNeighborsRegressor y KNeighborsClassifier. Como referencias de comparación se incluyeron DummyRegressor y DummyClassifier. Para el análisis no supervisado se utilizaron KMeans y PCA, este último como recurso de visualización en dos dimensiones.

También se evaluó SMOTENC, de la biblioteca imbalanced-learn, como alternativa para tratar el desbalance en clasificación. Esta estrategia se incorporó en un pipeline específico y se aplicó únicamente a los datos de entrenamiento de cada partición. Su evaluación no implicó adoptarla como tratamiento general de los modelos finales.

Métricas de evaluación

En regresión se utilizaron MAE, MSE, RMSE y R^2. MAE representa el error absoluto medio y se expresa en milisegundos; MSE corresponde al error cuadrático medio y se expresa en milisegundos cuadrados. RMSE es la raíz cuadrada de MSE, conserva la unidad de la variable objetivo y otorga mayor peso a los errores grandes. R^2 compara el error cuadrático del modelo con el de una predicción constante igual a la media de los valores observados en el conjunto evaluado: un valor de 1 indica predicción perfecta, 0 corresponde a esa referencia y un valor negativo indica un desempeño inferior. Se priorizó RMSE para seleccionar las configuraciones de regresión.

En clasificación se consideró la cancelación como clase positiva. Accuracy mide la proporción total de predicciones correctas; Precision indica qué proporción de las reservas predichas como canceladas realmente se canceló; y Recall expresa qué proporción de las cancelaciones reales fue detectada. F1 es la media armónica de Precision y Recall y se utilizó como métrica principal de selección. Balanced Accuracy promedia la sensibilidad de ambas clases, dando el mismo peso a canceladas y no canceladas. ROC-AUC evalúa la capacidad de discriminar entre las clases a través de distintos umbrales. La medida presentada como PR-AUC en las tablas se calculó mediante average_precision_score, que resume la relación entre Precision y Recall como precisión promedio —Average Precision— y no corresponde a una integración trapezoidal de la curva.

En K-Means se utilizaron la inercia, el coeficiente Silhouette y el índice Calinski-Harabasz. La inercia mide la suma de las distancias cuadradas de las observaciones a sus centroides y se examinó mediante el método del codo. Silhouette evalúa la cohesión de cada grupo y su separación respecto a los demás. Calinski-Harabasz relaciona la dispersión entre grupos con la dispersión dentro de ellos. Valores mayores de estos dos últimos indicadores favorecen una configuración al comparar agrupamientos sobre los mismos datos y con el mismo preprocesamiento.

RESULTADOS

Regresión

Resultados del Árbol de Decisión Regressor

La validación cruzada seleccionó `max_depth=8` y `min_samples_leaf=100`, con un RMSE_CV de 15.417. Para simplificar el árbol se eligió `ccp_alpha=0.05408` mediante la regla 1-SE.

En el bloque de código de la figura 14 se muestra la búsqueda de la estructura del árbol mediante `GridSearchCV`.

```
tree_grid = GridSearchCV(
    base_tree_pipe,
    param_grid=tree_param_grid,
    scoring=scoring,
    refit='RMSE',
    cv=cv5,
)
tree_grid.fit(X_tree_cv, y_tree_cv)
```

Figura 14. Selección de hiperparámetros del árbol mediante Cross-Validation.

En la figura 15 se muestra la evaluación de `ccp_alpha` mediante validación cruzada para seleccionar el nivel de poda.

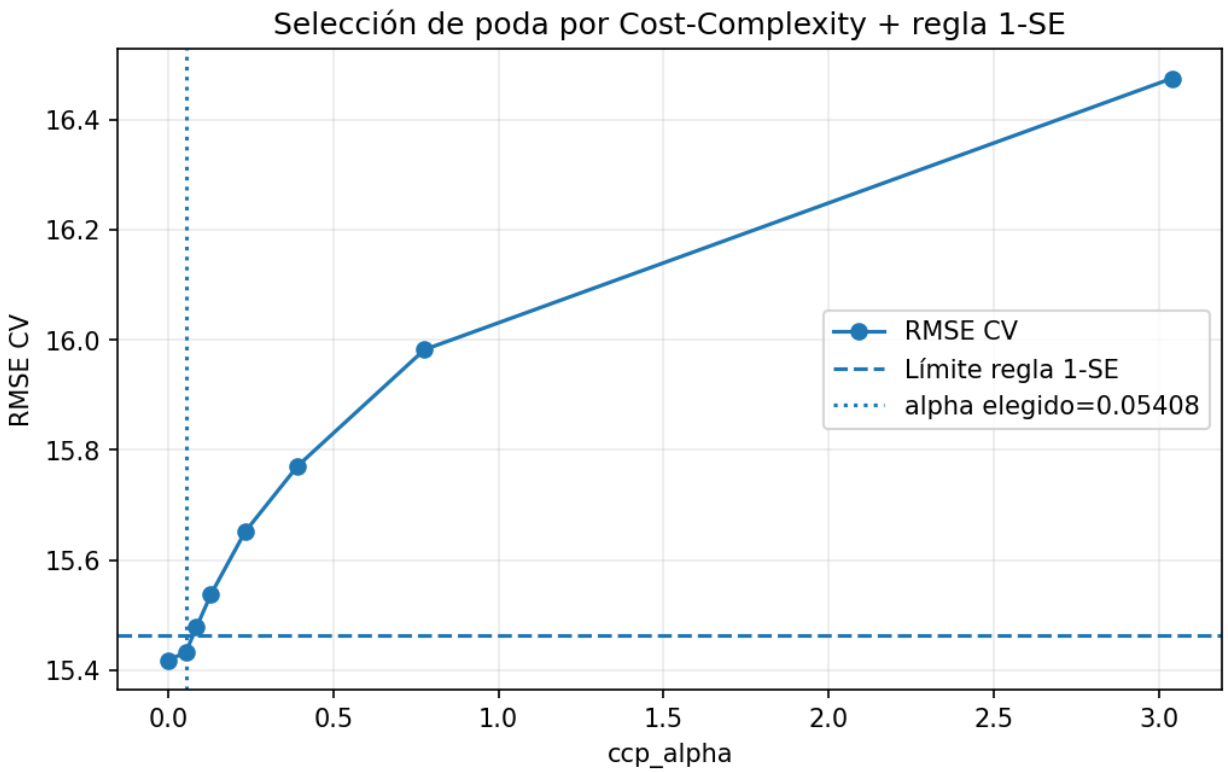


Figura 15. Selección del parámetro de poda del árbol.

Análisis de subajuste, ajuste óptimo y sobreajuste

N°	Modelo	fit_s	predict_s	MAE	MSE	RMSE	R^2
0	Subajuste, max_depth = 2	0.2602	0.0383	13.6655	293.8317	17.1415	0.1878
1	Árbol podado CV	0.7128	0.0405	12.2279	235.0786	15.3323	0.3502
2	Árbol sin poda	2.0267	0.0774	17.4171	477.0249	21.8409	−0.3186

Tabla 2. Comparación de los tres regímenes del árbol de decisión en TEST

Interpretación del árbol final y nodo raíz

El árbol final utiliza `Sleep_Hours` como raíz, con un umbral de 6.395 horas. Las variables más importantes son `Sleep_Hours`, `PlayTime_Hours` y `Age`.

En el bloque de código de la figura 16 se muestra el entrenamiento del árbol seleccionado y su evaluación sobre TEST.

```
1  tree_pruned.fit(X_train, y_train)
2  pred_tree_pruned = tree_pruned.predict(X_test)
```

Figura 16. Evaluación del árbol de decisión podado.

En la figura 17 se muestra la estructura del árbol de decisión podado y sus divisiones principales.

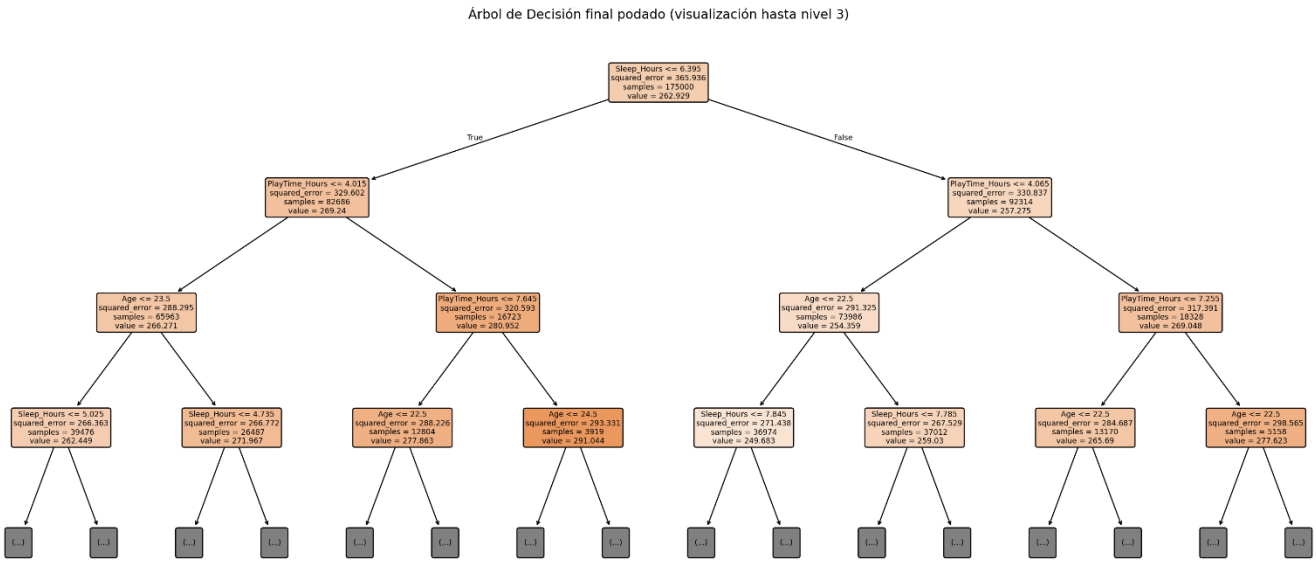


Figura 17. Visualización del árbol de decisión seleccionado.

Resultados del modelo Random Forest Regressor

El Random Forest se evaluó por etapas y posteriormente se compararon sus versiones podadas y sin poda.

Modelo	**fit_s**	**predict_s**	**MAE**	**MSE**	**RMSE**	**R^2**
Random Forest sin poda	34.9551	1.0594	12.2846	237.2989	15.4045	0.3441
Random Forest podado CV	11.2118	0.1765	12.0862	229.5079	15.1495	0.3656

Tabla 3. Efecto de la poda sobre Random Forest

En el bloque de código de la figura 18 se muestra la primera etapa de búsqueda del Random Forest, donde se evalúan la profundidad y el tamaño mínimo de las hojas.

```python
rf_struct_pipe = Pipeline([
    ('preprocessor', make_tree_preprocessor()),
    ('model', RandomForestRegressor(
        n_estimators=25,
        bootstrap=True,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )),
])

rf_struct_grid = GridSearchCV(
    rf_struct_pipe,
    param_grid={
        'model__max_depth': [8, 12, None],
        'model__min_samples_leaf': [1, 50, 100],
    },
    scoring=scoring,
    refit='RMSE',
    cv=cv5,
    n_jobs=1,
)
rf_struct_grid.fit(X_rf_cv, y_rf_cv)
```

Figura 18. Selección de la estructura inicial del Random Forest.

Número de árboles	Entrenamiento (s)	RMSE TEST (ms)	R^2 TEST
10	2.2253	15.1622	0.3646
50	10.6100	15.1495	0.3656
150	31.5282	15.1456	0.3659

Tabla 4. Variantes de Random Forest según el número de árboles

El aumento de 10 a 150 árboles redujo ligeramente el RMSE, de 15.1622 a 15.1456 ms, mientras que el tiempo de entrenamiento pasó de 2.2253 a 31.5282 segundos. En esta comparación, el incremento del costo computacional fue considerable frente a la pequeña mejora

predictiva. Los tiempos corresponden a la ejecución registrada y pueden variar según el equipo y las condiciones de procesamiento.

En la figura 19 se muestra un árbol representativo perteneciente al Random Forest seleccionado.

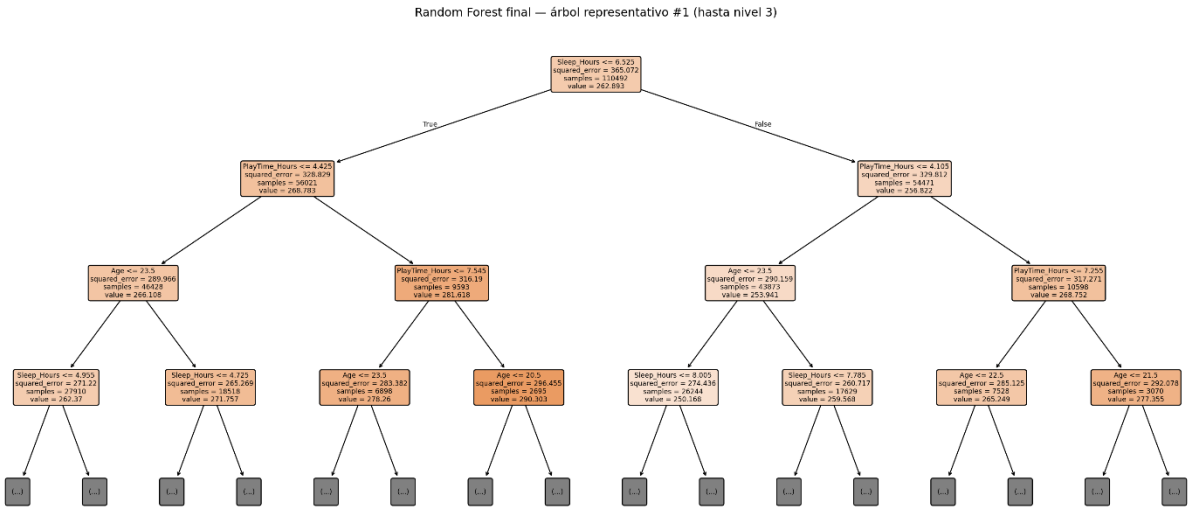


Figura 19. Estructura de uno de los árboles del ensamble.

En la Figura 20 se muestra el tiempo de entrenamiento de las variantes de Random Forest con 10, 50 y 150 árboles. Sus resultados predictivos se presentan en la Tabla 4.

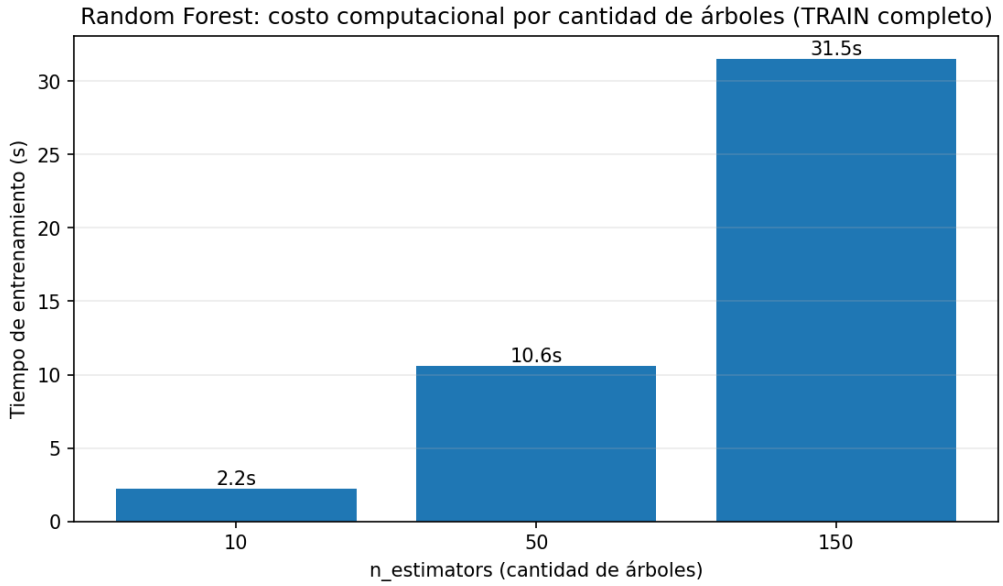


Figura 20. Tiempo de entrenamiento según el número de árboles en Random Forest.

Resultados del modelo KNN Regressor

KNN se evaluó utilizando variables numéricas escaladas y diferentes valores de `K` y pesos. En el bloque de código de la figura 21 se muestra la búsqueda conjunta del número de vecinos y del esquema de pesos de KNN.

```python
knn_grid = GridSearchCV(
    knn_pipe,
    param_grid={
        'model__n_neighbors': [5, 10, 20, 30, 50, 70, 100, 150],
        'model__weights': ['uniform', 'distance'],
    },
    scoring=scoring,
    refit='RMSE',
    cv=cv5,
    n_jobs=1,
)
knn_grid.fit(X_knn_cv, y_knn_cv)
```

Figura 21. GridSearchCV de KNN.

En la figura 22 se muestra la selección de `K` y del esquema de pesos mediante validación cruzada.

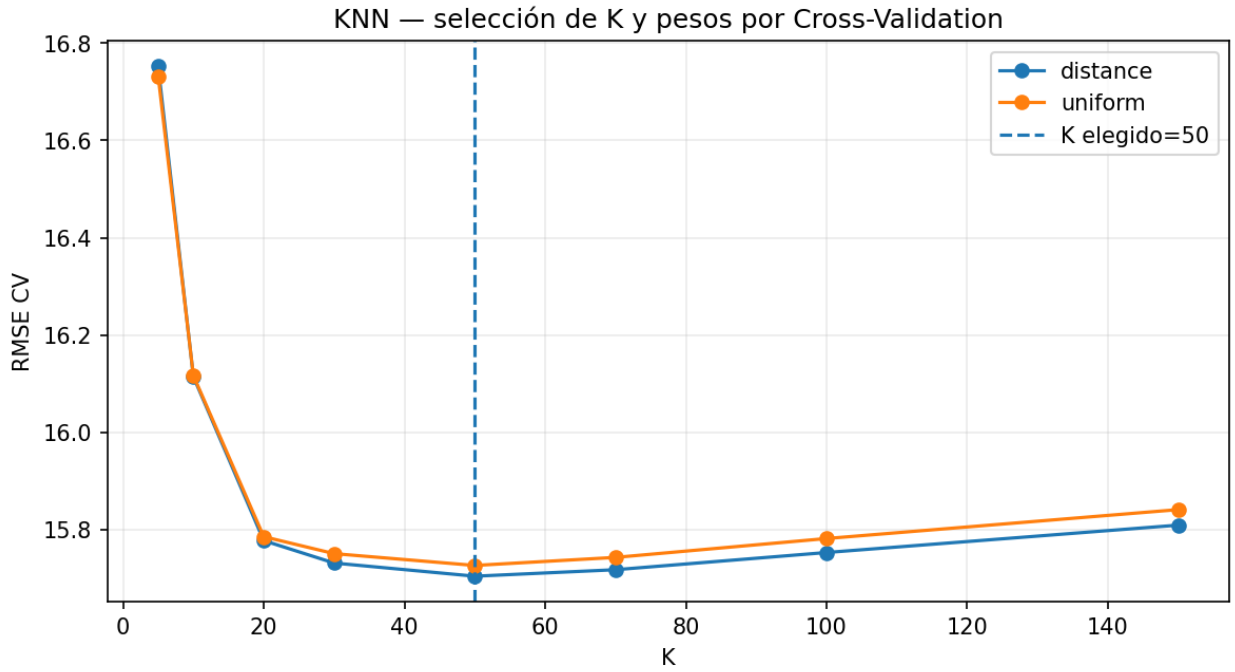


Figura 22. RMSE de CV para los valores de K evaluados.

Selección del número de vecinos

La selección del número de vecinos se realizó mediante validación cruzada de cinco particiones sobre una muestra reproducible de 8.000 registros de TRAIN. Se evaluaron los valores K ∈ {5, 10, 20, 30, 50, 70, 100, 150}, combinados con pesos uniformes y pesos por

distancia. Con ponderación por distancia, el RMSE medio disminuyó de 16.752 ms para K = 5 a 15.777 ms para K = 20 y alcanzó su mínimo en K = 50, con 15.704 ms. Para valores superiores, el error aumentó gradualmente. Por tanto, se seleccionó K = 50 con weights = 'distance'.

Este comportamiento es compatible con el equilibrio entre sensibilidad a las variaciones locales y suavizado de las predicciones: pocos vecinos pueden hacer al modelo más sensible al ruido, mientras que demasiados pueden diluir diferencias relevantes entre observaciones. Al entrenar sobre TRAIN completo, la comparación adicional de pesos para K = 50 produjo un RMSE en TEST de 15.297 ms con pesos uniformes y 15.292 ms con pesos por distancia. La diferencia fue pequeña; la selección de la configuración se sustentó en la validación cruzada.

Visualización del vecindario y efecto del tamaño de entrenamiento

El RMSE de 15.704 ms corresponde a la búsqueda de hiperparámetros mediante validación cruzada sobre 8.000 registros de TRAIN, mientras que el RMSE de 15.292 ms corresponde a la evaluación en TEST del modelo entrenado con 175.000 registros. Estos resultados proceden de tamaños de entrenamiento y conjuntos de evaluación distintos, por lo que la diferencia no puede atribuirse exclusivamente al aumento de datos. La Figura 23 muestra el vecindario utilizado para una observación consultada, proyectado en dos dimensiones para facilitar su interpretación.

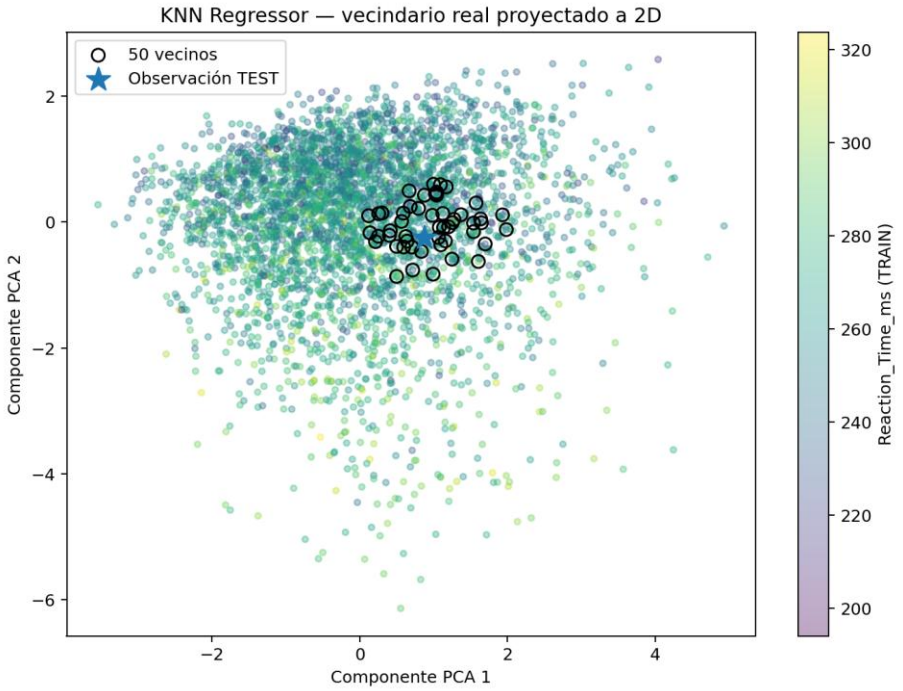


Figura 23. Observación consultada y sus vecinos más cercanos.

Comparación final de los modelos

Evaluación sobre el mismo conjunto TEST

Finalmente, los tres modelos se compararon utilizando el mismo conjunto TEST y las mismas métricas.

Modelo	**MAE**	**MSE**	**RMSE**	**R^2**	**Reducción del RMSE frente al baseline (%)**
Random Forest podado CV	12.0862	229.5079	15.1495	0.3656	20.3526
KNN K = 50, distance	12.1874	233.8547	15.2923	0.3536	19.6019
Árbol podado CV	12.2279	235.0786	15.3323	0.3502	19.3918
Baseline, media	15.1553	361.7885	19.0207	−0.000024	0.0000

Tabla 5. Comparación final de los modelos sobre el mismo conjunto TEST

Comparación común mediante Cross-Validation

Modelo	**RMSE CV, media ± desviación estándar**	**MAE CV medio**	**R^2 CV medio**
Random Forest podado CV	15.3361 ± 0.3344	12.2590	0.3650
KNN K = 50, distance	15.6981 ± 0.2967	12.4916	0.3346
Árbol podado CV	15.8501 ± 0.3087	12.6504	0.3216
Baseline, media	19.2487 ± 0.3250	15.3944	−0.0005

Tabla 6. Comparación común mediante Cross-Validation

Los modelos se evaluaron sobre una muestra común de 8.000 registros de TRAIN y utilizando las mismas cinco particiones de validación cruzada. MAE y RMSE se expresan en milisegundos.

En la figura 24 se muestra la comparación del RMSE obtenido por los tres algoritmos y el baseline sobre el mismo conjunto TEST.

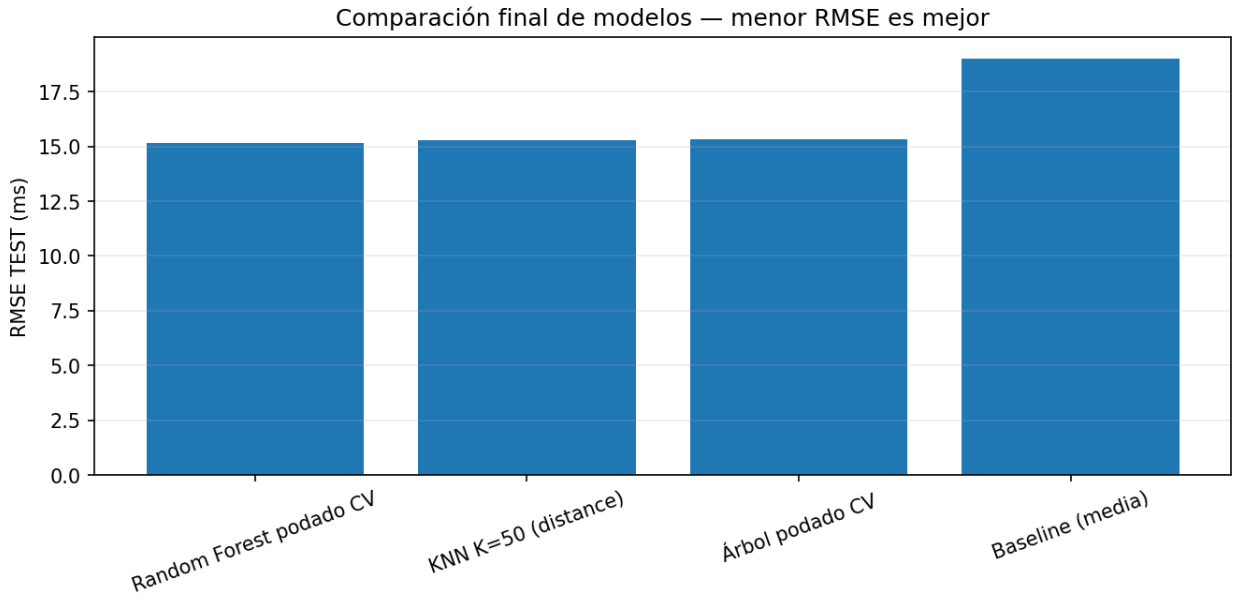


Figura 24. Comparación final del error RMSE.

En la figura 25 se muestra la comparación de los modelos mediante el coeficiente de determinación R^2.

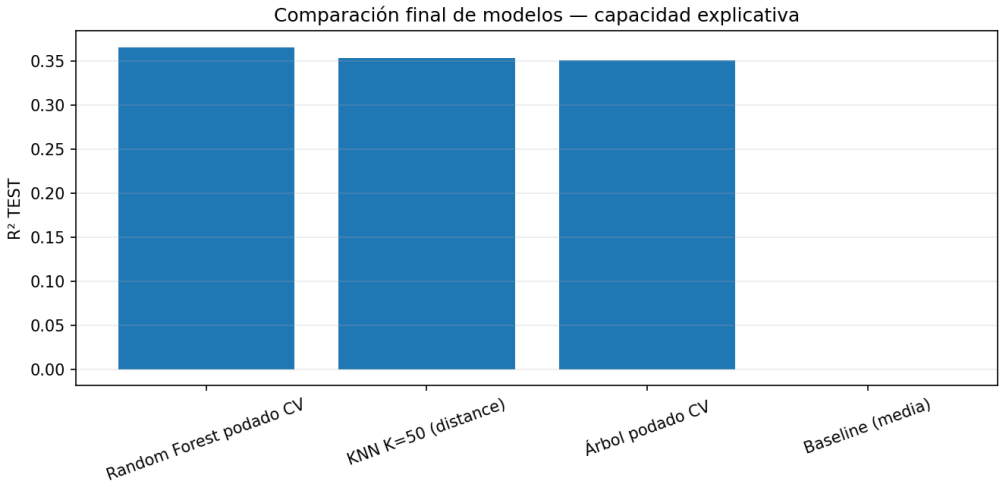


Figura 25. Comparación final del R^2.

En la figura 26 se muestra la comparación común de los tres modelos usando los mismos folds de validación cruzada.

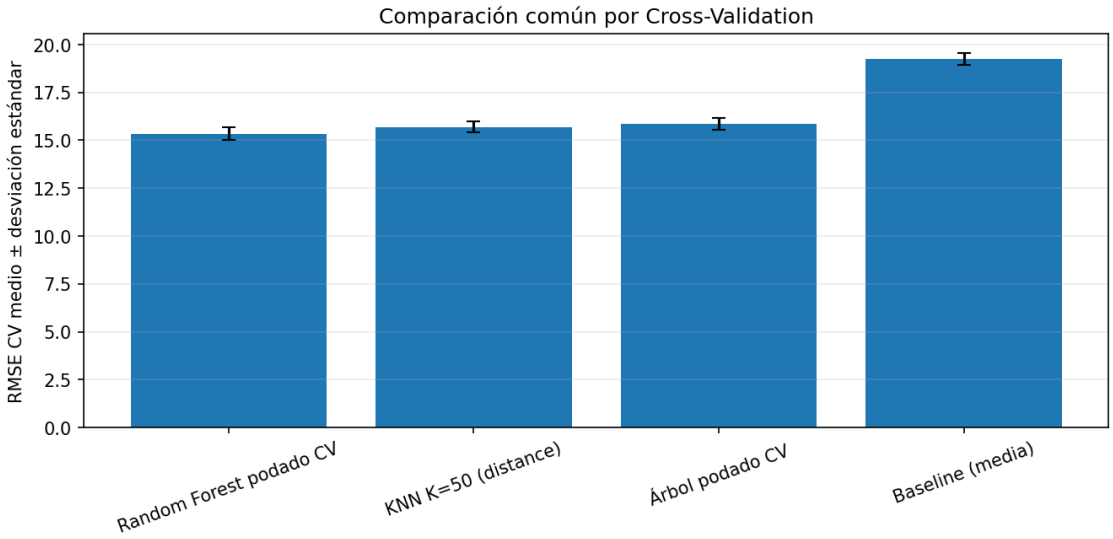


Figura 26. RMSE de CV de las configuraciones finales.

Diagnóstico de residuos de los modelos finales

En la figura 27 se muestra el análisis de residuos del árbol de decisión podado.

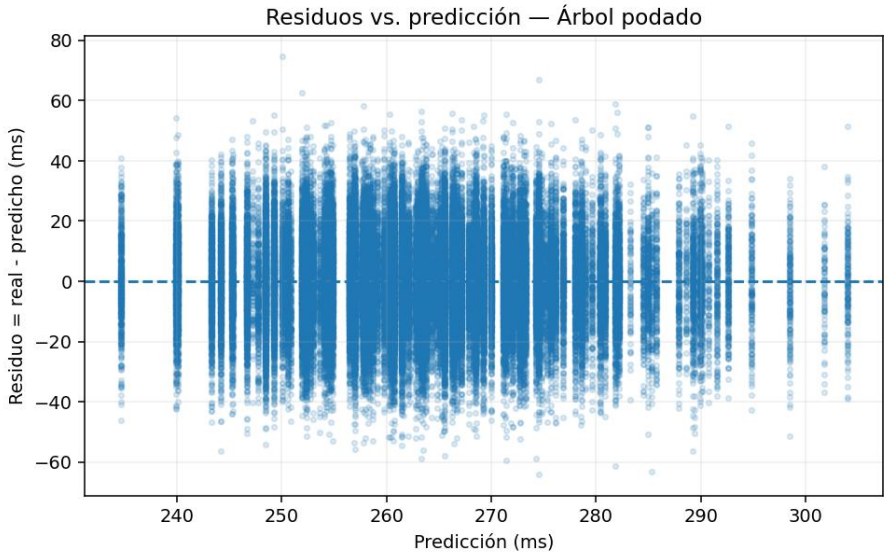


Figura 27. Diagnóstico de residuos del árbol podado.

En la figura 28 se muestra el análisis de residuos del modelo KNN.

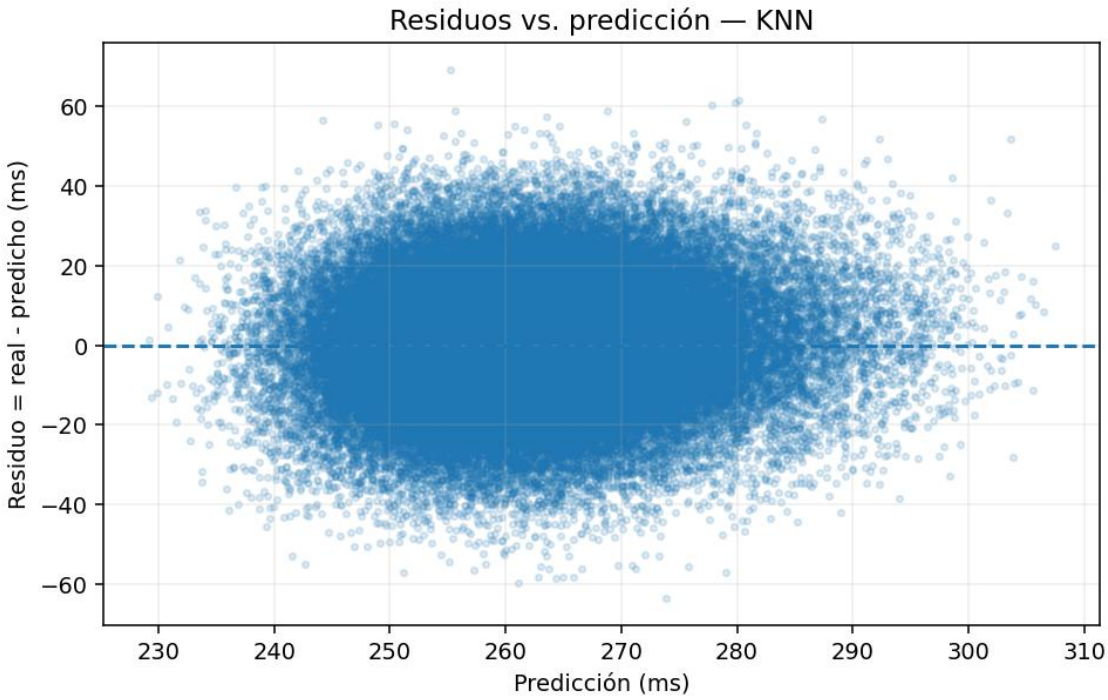


Figura 28. Diagnóstico de residuos de KNN.

En la figura 29 se muestra el análisis de residuos del Random Forest podado.

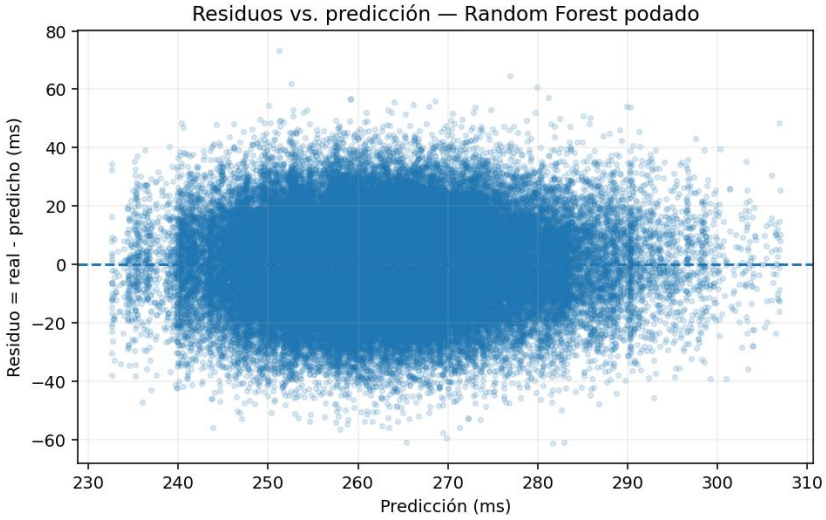


Figura 29. Diagnóstico de residuos del Random Forest podado.

Clasificación

Clasificación con Árbol de Decisión

El árbol de decisión fue entrenado mediante búsqueda de hiperparámetros utilizando validación cruzada. Se evaluaron diferentes profundidades, tamaños mínimos de hojas y niveles de poda con el objetivo de encontrar una estructura capaz de capturar patrones del conjunto de datos sin memorizar ejemplos específicos.

El control de complejidad fue especialmente importante en este modelo debido a que los árboles sin restricciones pueden generar divisiones muy específicas que reducen su capacidad de generalización.

Para evitar árboles demasiado complejos se evaluó el parámetro de poda mediante validación cruzada. La Figura 30 muestra el comportamiento del modelo frente a diferentes valores de complejidad.

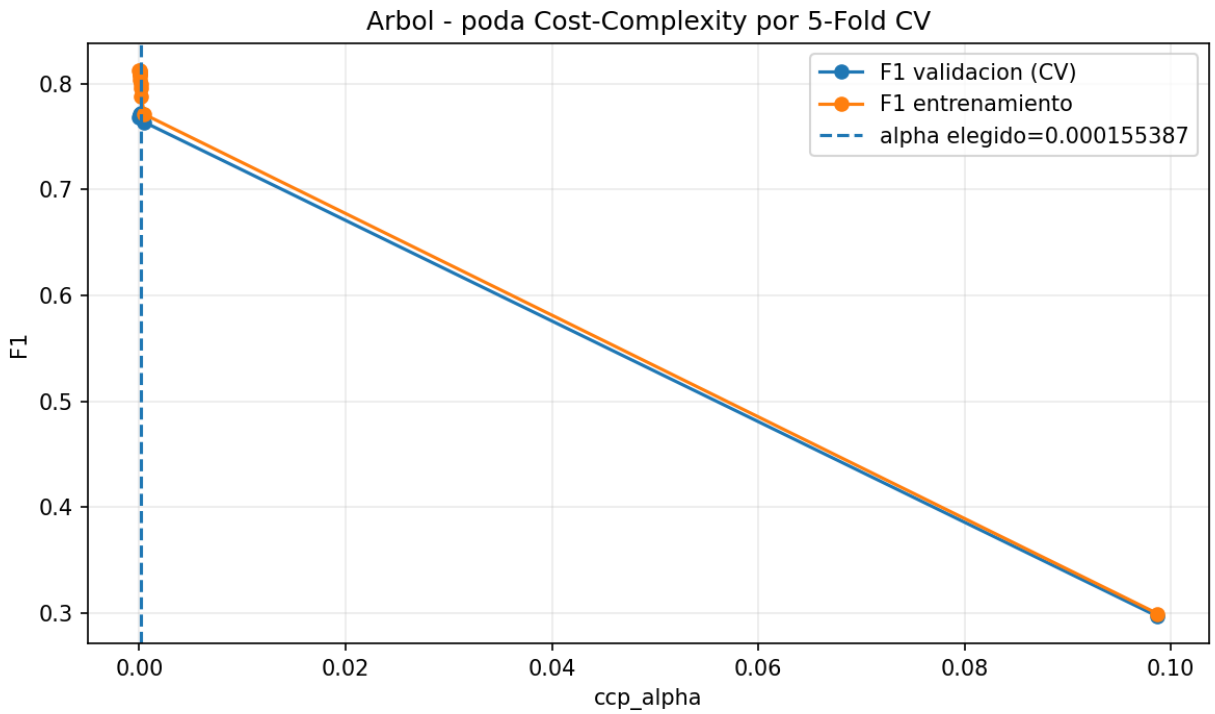


Figura 30. Selección del parámetro de poda del Árbol de Decisión mediante Cross-Validation.

Ajuste de los umbrales de clasificación

Los umbrales de clasificación se ajustaron maximizando F1 sobre probabilidades obtenidas mediante validación cruzada dentro de TRAIN. Cada probabilidad se calculó para una observación que no participó en el entrenamiento del modelo que la produjo. Los umbrales seleccionados fueron 0.58 para el árbol podado, 0.49 para Random Forest y 0.43 para KNN. Estos valores se utilizaron para obtener las etiquetas predichas en la evaluación final sobre TEST. Las métricas ROC-AUC y Average Precision se calcularon a partir de las probabilidades, sin aplicar esos cortes.

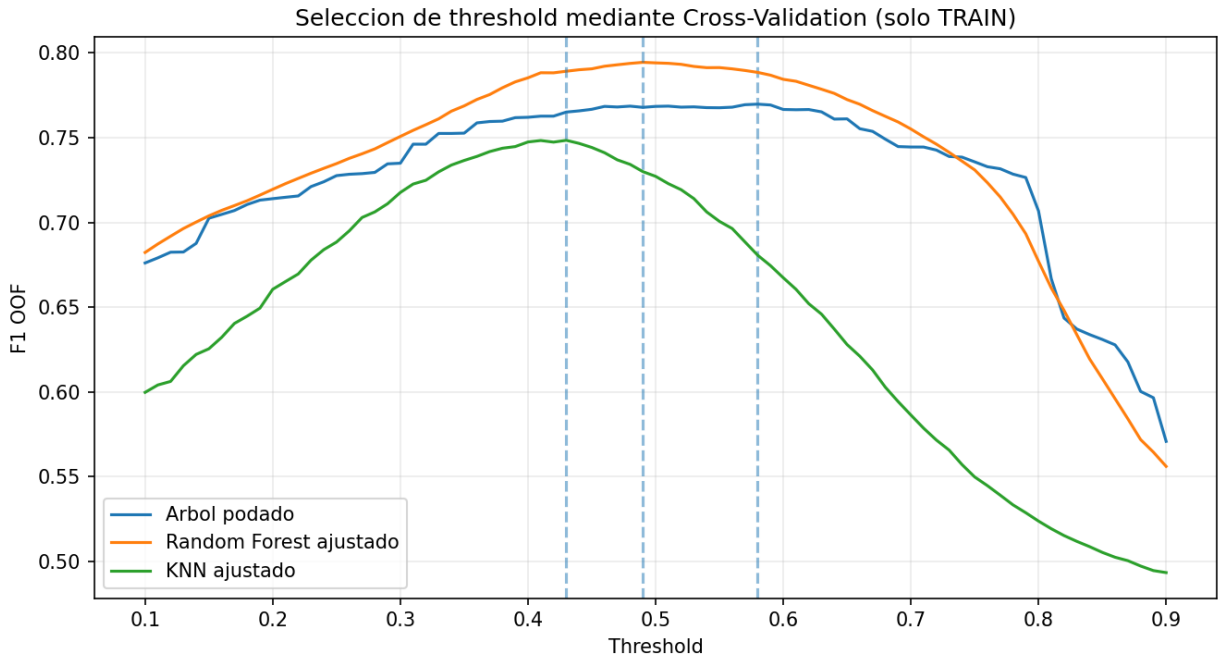


Figura 31. Selección de umbrales mediante validación cruzada en TRAIN

La configuración seleccionada correspondió a un árbol podado con ajuste del umbral de clasificación en 0.58. En el conjunto TEST obtuvo los siguientes resultados como se observa en la tabla 7:

Métrica	**Resultado**
Accuracy	0.8362
Balanced Accuracy	0.8172

Precision	0.7999
Recall	0.7438
F1-score	0.7708
ROC-AUC	0.9127
PR-AUC	0.8691

Tabla 7. Resultados TEST árbol de decisión

El modelo presenta un comportamiento equilibrado entre la detección de cancelaciones y la reducción de falsos positivos. La poda permite disminuir la complejidad del árbol manteniendo una capacidad predictiva adecuada, evitando que las reglas generadas dependan excesivamente de características particulares del conjunto de entrenamiento.

Como se muestra en la Figura 32, la estructura final obtenida permite observar las principales divisiones realizadas por el modelo y la forma en que las variables seleccionadas contribuyen al proceso de clasificación.

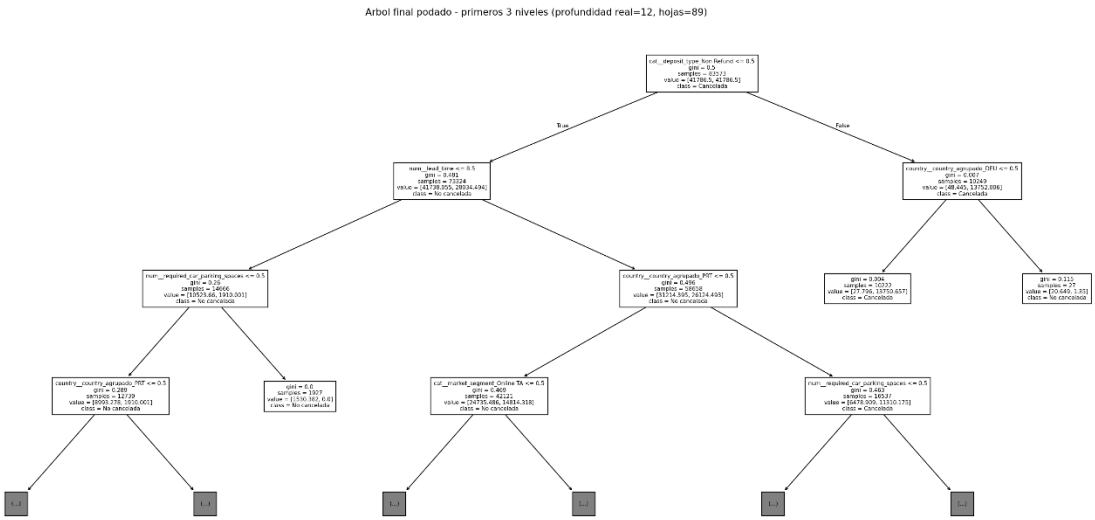


Figura 32. Estructura del Árbol de Decisión clasificador seleccionado.

Además de las métricas globales, se analizó la distribución de aciertos y errores mediante la matriz de confusión. La Figura 33 permite identificar la cantidad de registros correctamente clasificados y aquellos casos donde el modelo presentó errores.

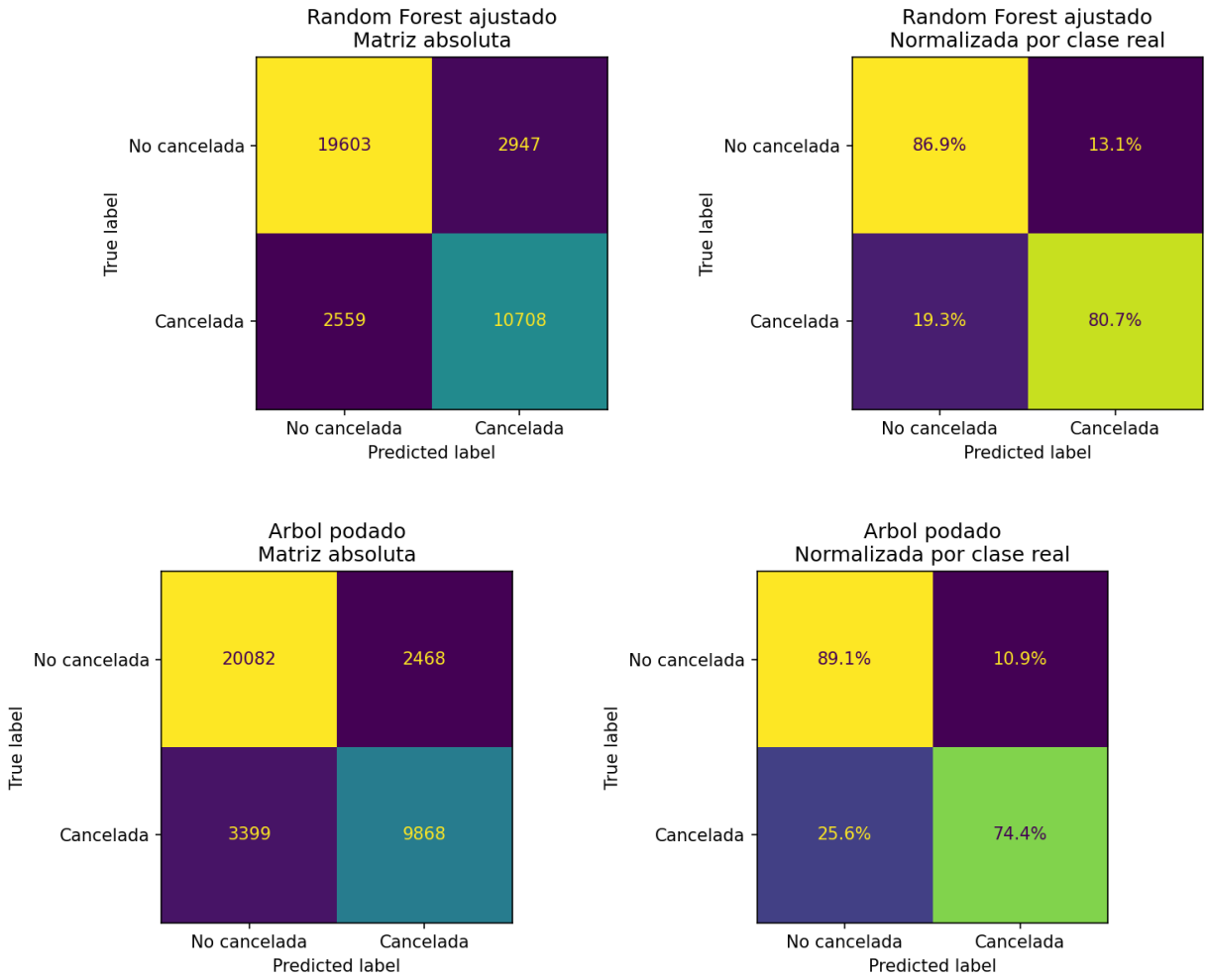


Figura 33. Matriz de confusión del Árbol de Decisión clasificador.

Clasificación con Random Forest

El modelo Random Forest fue evaluado como una alternativa basada en ensamble de múltiples árboles de decisión. La principal ventaja de este algoritmo consiste en combinar diferentes árboles entrenados sobre muestras y subconjuntos de variables distintos, reduciendo la variabilidad asociada a un único árbol.

Se evaluaron diferentes configuraciones relacionadas con la profundidad, cantidad de árboles y parámetros de regularización. La configuración seleccionada correspondió a un Random Forest ajustado mediante validación cruzada.

Para analizar el efecto del tamaño del ensamble se evaluaron diferentes cantidades de árboles. La Figura 34 muestra la relación entre el número de estimadores y el costo computacional asociado.

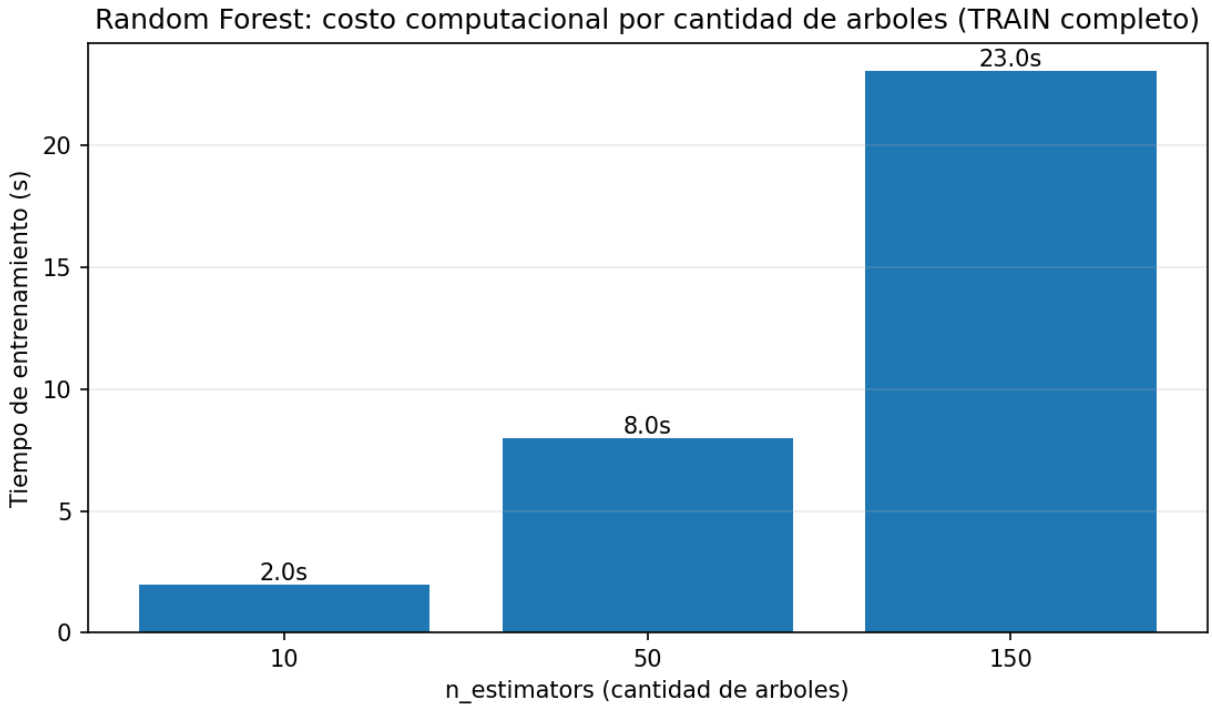


Figura 34. Tiempo de entrenamiento según número de árboles en Random Forest.

En el conjunto TEST obtuvo los siguientes resultados, como se observa en la tabla 8:

Métrica	**Resultado**
Accuracy	0.8463
Balanced Accuracy	0.8382
Precision	0.7842
Recall	0.8071
F1-score	0.7955
ROC-AUC	0.9288

PR-AUC	0.8986

Tabla 8. Resultados TEST random forest

Frente al Árbol de Decisión podado, Random Forest presenta mejores valores en Accuracy, Balanced Accuracy, F1-score y ROC-AUC. La diferencia no es muy amplia, pero sí se mantiene de forma consistente en las principales métricas evaluadas. Aunque Random Forest está compuesto por múltiples árboles, la visualización de un árbol representativo permite comprender la estructura interna utilizada por el ensamble.

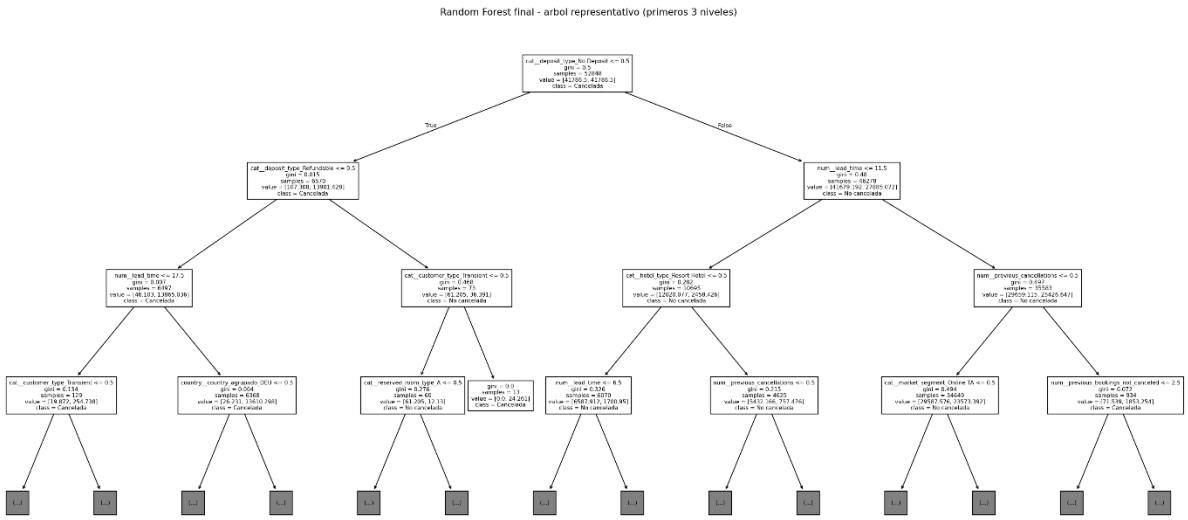


Figura 35. Árbol representativo perteneciente al modelo Random Forest.

El análisis de importancia de variables mostró que las características con mayor influencia fueron:

- deposit_type_Non Refund
- lead_time
- país del cliente
- deposit_type_No Deposit
- market_segment
- total_of_special_requests

Estos resultados indican que las condiciones económicas y características del proceso de reserva tienen una relación importante con la probabilidad de cancelación.

Para interpretar el comportamiento del modelo se analizó la importancia relativa de las variables utilizadas. La Figura 36 presenta las características con mayor contribución dentro de la clasificación.

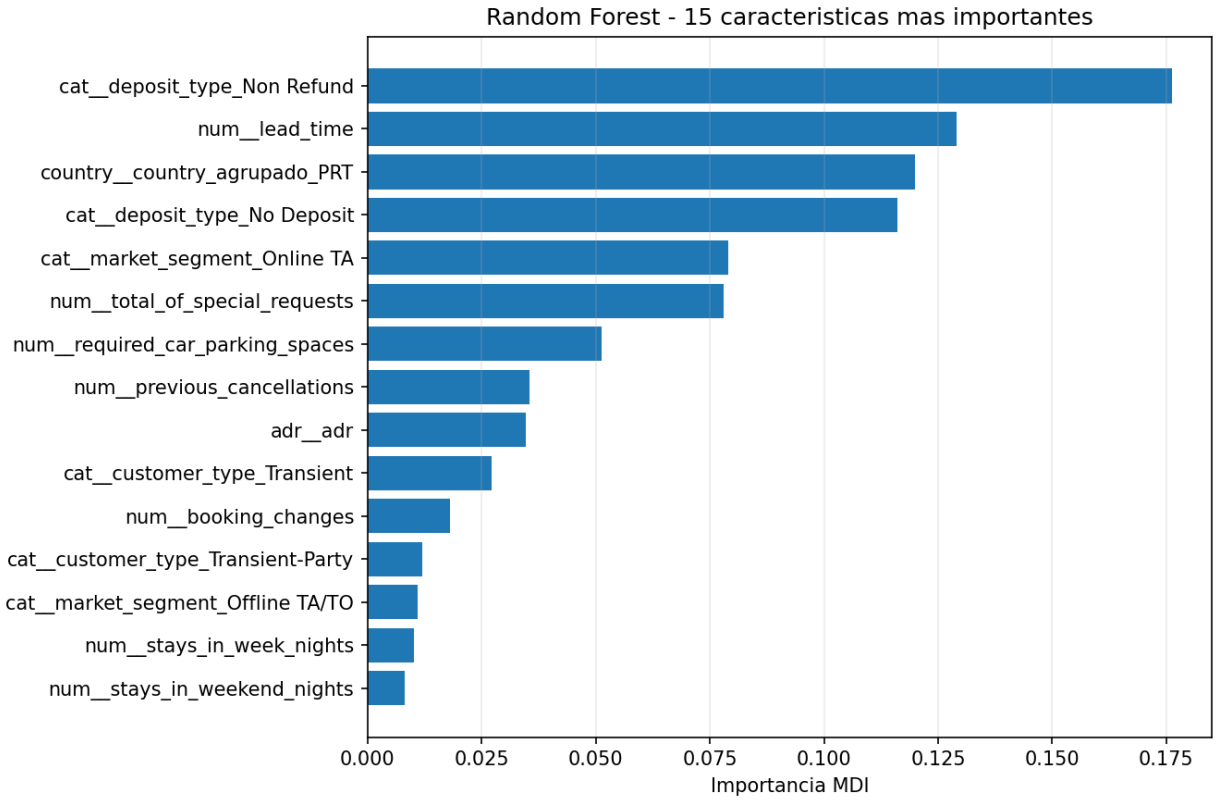


Figura 36. Importancia de variables del modelo Random Forest clasificador.

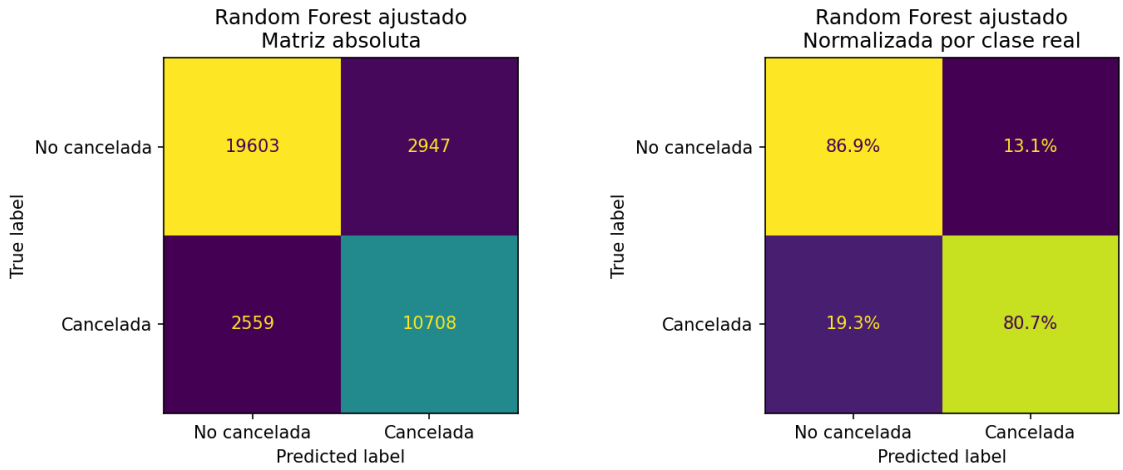


Figura 37. Matriz de confusión del Random Forest clasificador ajustado.

La capacidad de discriminación de los tres clasificadores se evaluó mediante sus curvas ROC sobre TEST. La figura 38 compara la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos para distintos umbrales de decisión.

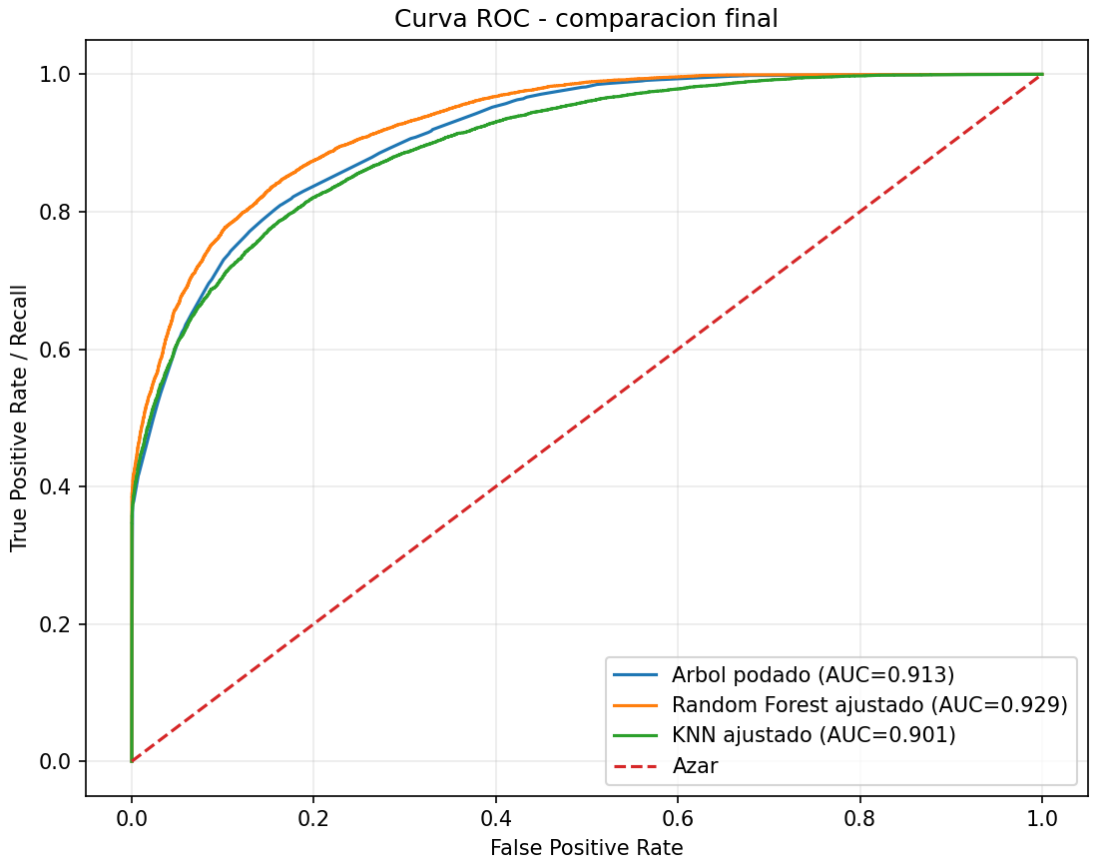


Figura 38. Curvas ROC de los tres clasificadores sobre TEST.

Clasificación con KNN

El modelo KNN fue evaluado utilizando variables escaladas y diferentes valores de vecinos. Debido a que este algoritmo clasifica una observación según la similitud con otros registros cercanos, la normalización de variables resulta necesaria para evitar que atributos con escalas mayores dominen el cálculo de distancia.

Se realizó una búsqueda conjunta mediante validación cruzada estratificada de cinco particiones sobre una muestra de 12.000 registros de TRAIN. Se evaluaron $K \in \{3, 5, 7, 10, 15, 20, 30, 50, 75, 100\}$, pesos uniformes o por distancia y las distancias Manhattan ($p = 1$) y euclidiana ($p = 2$). La configuración con mayor F1 medio de validación cruzada fue $K = 30$, weights = 'distance' y $p = 1$, con $F1 = 0.7115$. Esta configuración se utilizó para entrenar el modelo final sobre TRAIN completo. El valor seleccionado representa el mejor resultado dentro de la búsqueda realizada.

Debido a que KNN depende directamente de la distancia entre observaciones, la selección adecuada del número de vecinos resulta importante para evitar modelos demasiado sensibles o demasiado generales. La Figura 39 muestra la evaluación realizada mediante validación cruzada.

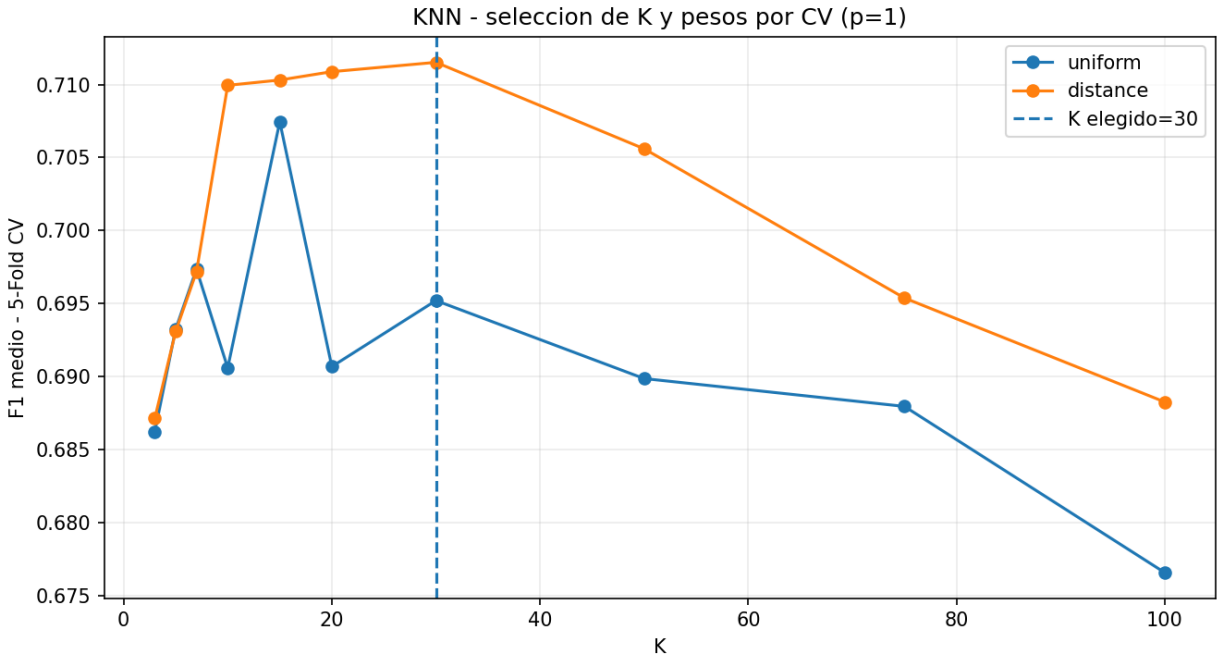


Figura 39. Evaluación del parámetro K mediante Cross-Validation en KNN.

Como se muestra en la tabla 9 los resultados finales fueron:

Métrica	**Resultado**
Accuracy	0.8235
Balanced Accuracy	0.8086
Precision	0.7674
Recall	0.7512
F1-score	0.7592
ROC-AUC	0.9014
PR-AUC	0.8696

Tabla 9. Resultados KNN

Aunque KNN obtuvo resultados competitivos, presentó un mayor costo durante la predicción debido a que debe calcular distancias contra múltiples observaciones almacenadas. En

comparación con los modelos basados en árboles, su tiempo de inferencia fue considerablemente mayor, lo que representa una limitación cuando se requiere clasificar grandes volúmenes de registros.

La Figura 40 muestra la distribución del vecindario utilizado por KNN para una observación específica, permitiendo interpretar cómo el modelo determina la clasificación a partir de registros similares.

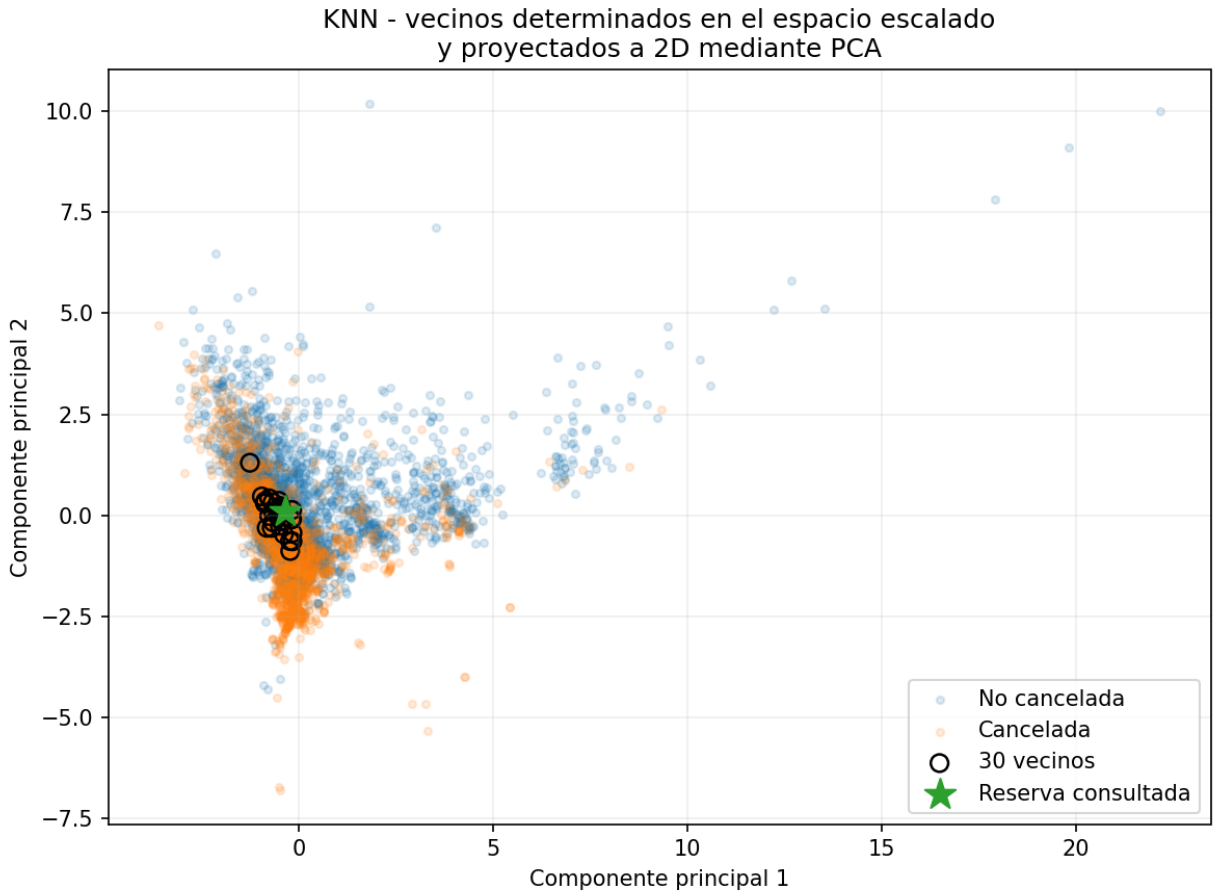


Figura 40. Vecindario utilizado por KNN para una observación consultada.

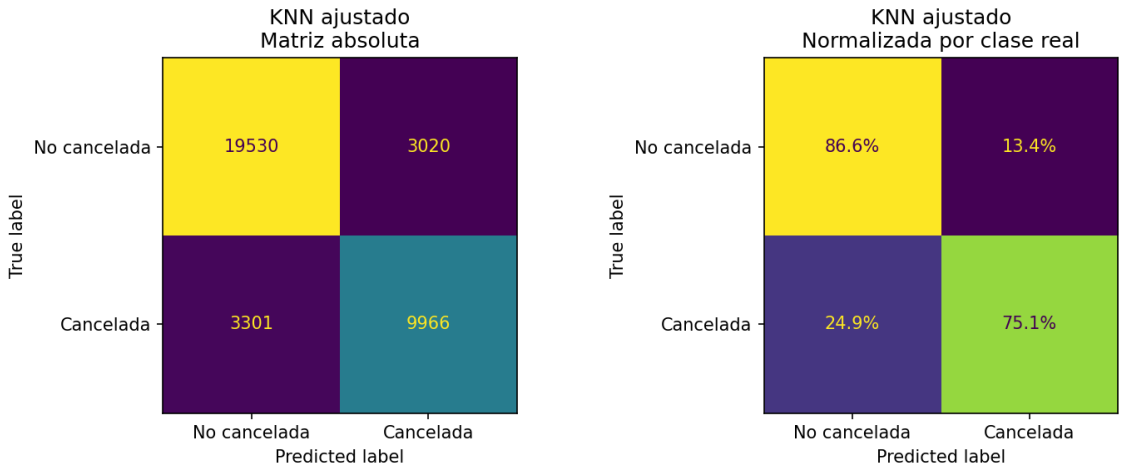


Figura 41. Matriz de confusión del KNN.

Comparación final de modelos

Los tres modelos fueron comparados utilizando el mismo conjunto TEST y las mismas métricas para garantizar una evaluación justa, la tabla 10 muestra esta comparación:

Modelo	**Accuracy**	**Balanced Accuracy**	**F1**	**ROC-AUC**
Random Forest ajustado	0.8463	0.8382	0.7955	0.9288
Árbol podado	0.8362	0.8172	0.7708	0.9127
KNN ajustado	0.8235	0.8086	0.7592	0.9014

Tabla 10. Comparación modelos

Los resultados muestran diferencias moderadas entre los modelos. Random Forest presenta el comportamiento más estable debido a su capacidad para combinar múltiples estructuras de decisión y reducir la variabilidad propia de un árbol individual. Sin embargo, el árbol podado mantiene un rendimiento cercano con una estructura más sencilla y mayor facilidad de interpretación.

KNN logró identificar patrones presentes en los datos, aunque su costo computacional durante la predicción limita su aplicación cuando se requiere procesar grandes cantidades de reservas.

Finalmente, los modelos fueron comparados utilizando las mismas métricas sobre el conjunto TEST. La Figura 42 resume visualmente el comportamiento obtenido por cada algoritmo.

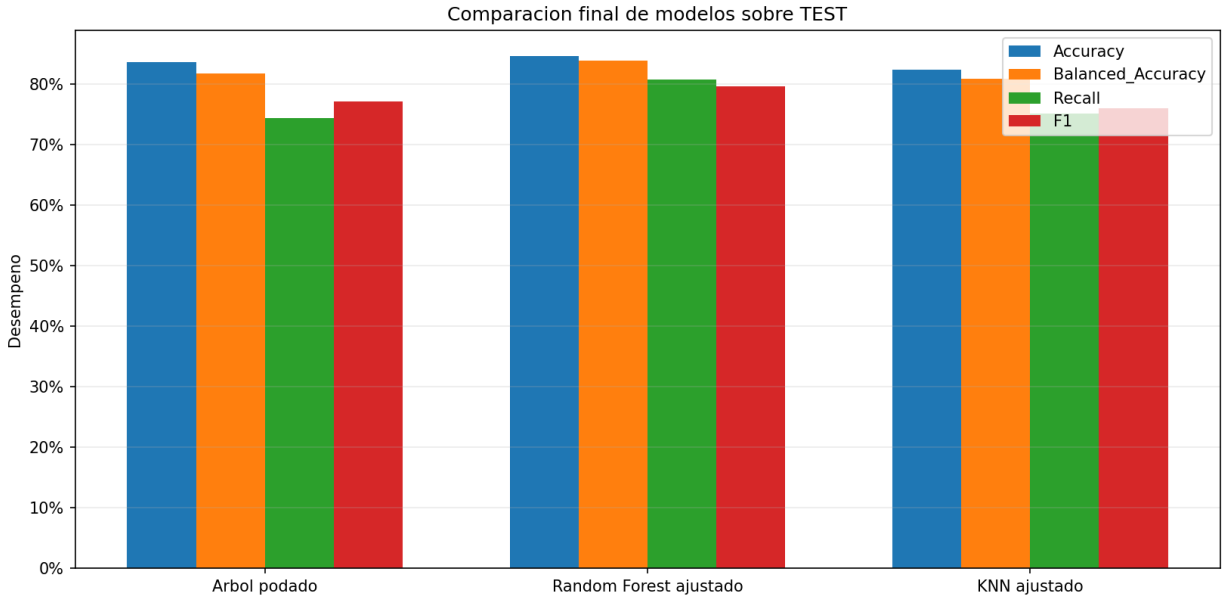


Figura 42. Comparación final del desempeño de los modelos de clasificación.

Análisis no supervisado mediante K-Means

Como complemento al análisis supervisado se aplicó el algoritmo K-Means con el propósito de identificar posibles agrupamientos naturales dentro de los registros de reservas hoteleras y observar si dichos grupos presentan relación con la variable de cancelación.

La selección del número de grupos se evaluó mediante diferentes criterios. El análisis del coeficiente Silhouette y del índice Calinski-Harabasz mostró que la configuración con dos grupos presenta la mejor separación interna de los datos.

Para determinar una cantidad adecuada de agrupamientos se utilizó el método del codo. Como se observa en la Figura 43, este análisis permitió evaluar diferentes configuraciones de

clusters.

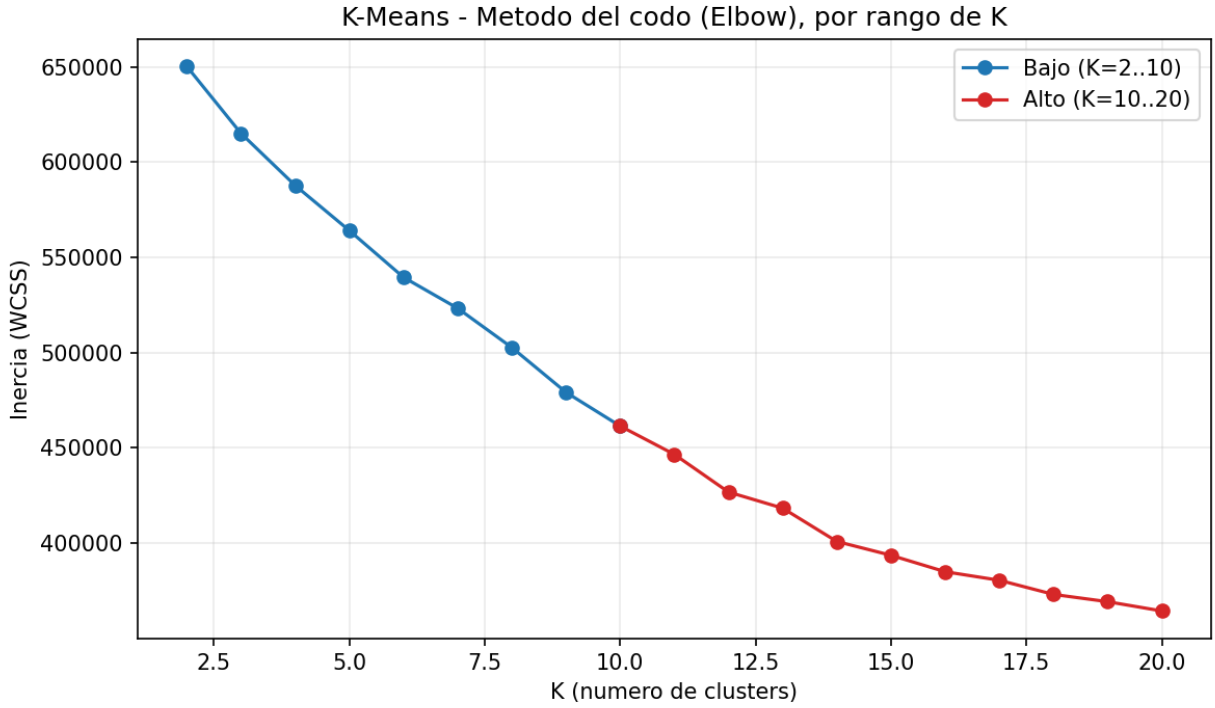


Figura 43. Método del codo para selección del número de clusters.

Complementariamente se evaluó la separación interna de los grupos mediante el coeficiente Silhouette.

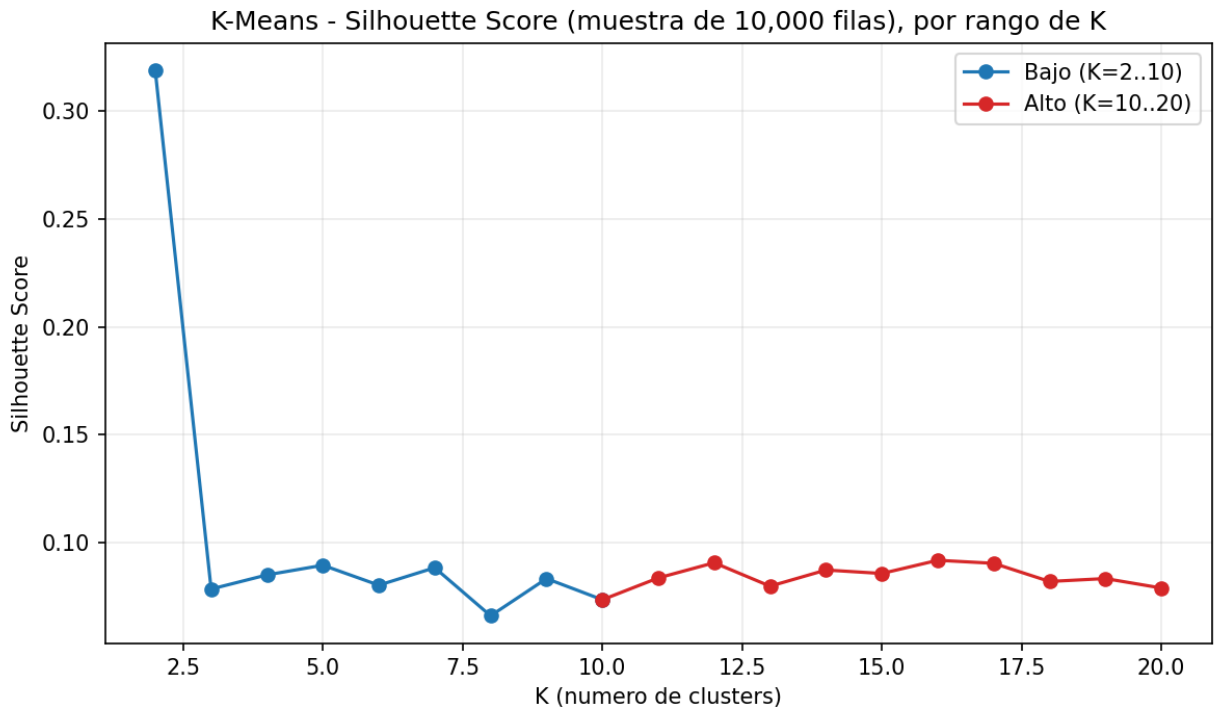


Figura 44. Evaluación Silhouette de los clusters obtenidos mediante K-Means.

La tabla 11 muestra la comparación del número de grupos sugerido por cada criterio:

Rango evaluado	**Codo aproximado**	**Mejor K según Silhouette**	**Mejor K según Calinski-Harabasz**
K = 2–10	6	2; valor = 0.3186	2; valor = 2572.84
K = 10–20	14	16; valor = 0.0919	12; valor = 1783.12

Tabla 11. Comparación del número de grupos sugerido por cada criterio

Se compararon dos rangos del número de grupos: K = 2–10 y K = 10–20. La aproximación al método del codo sugirió K = 6 y K = 14, respectivamente. Silhouette y Calinski-Harabasz coincidieron en seleccionar K = 2 dentro del rango bajo. En el rango alto, sus mejores valores correspondieron a K = 16 y K = 12, respectivamente, y fueron inferiores a los obtenidos con K = 2. Por ello, se adoptaron dos grupos, atendiendo a la coincidencia de ambos indicadores. El método del codo aportó una sugerencia distinta, por lo que no hubo acuerdo entre los tres criterios.

El número seleccionado coincide con las dos clases de is_canceled, pero esa coincidencia numérica no demuestra que los grupos representen reservas canceladas y no canceladas. Para examinarlo, se compararon los agrupamientos con las etiquetas reales de las 30.000 reservas utilizadas en este análisis.

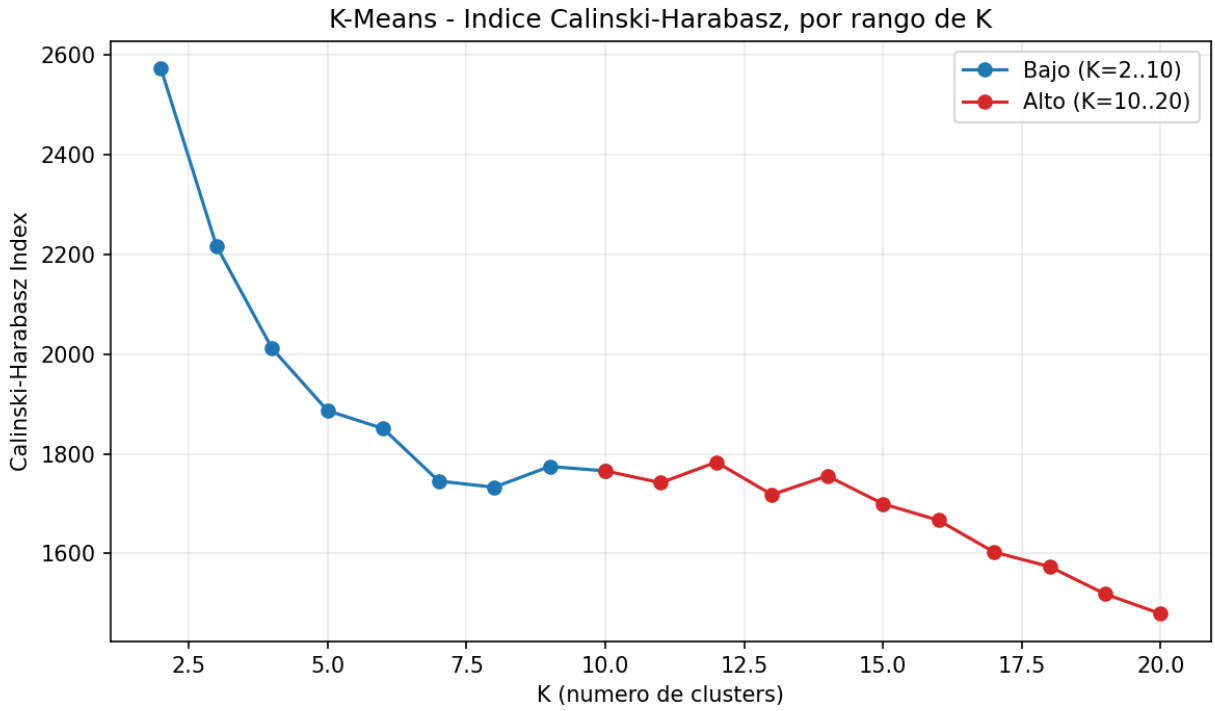


Figura 45. Índice Calinski-Harabasz por rango de K

Finalmente, se realizó una reducción dimensional mediante PCA para visualizar la distribución espacial de los grupos encontrados.

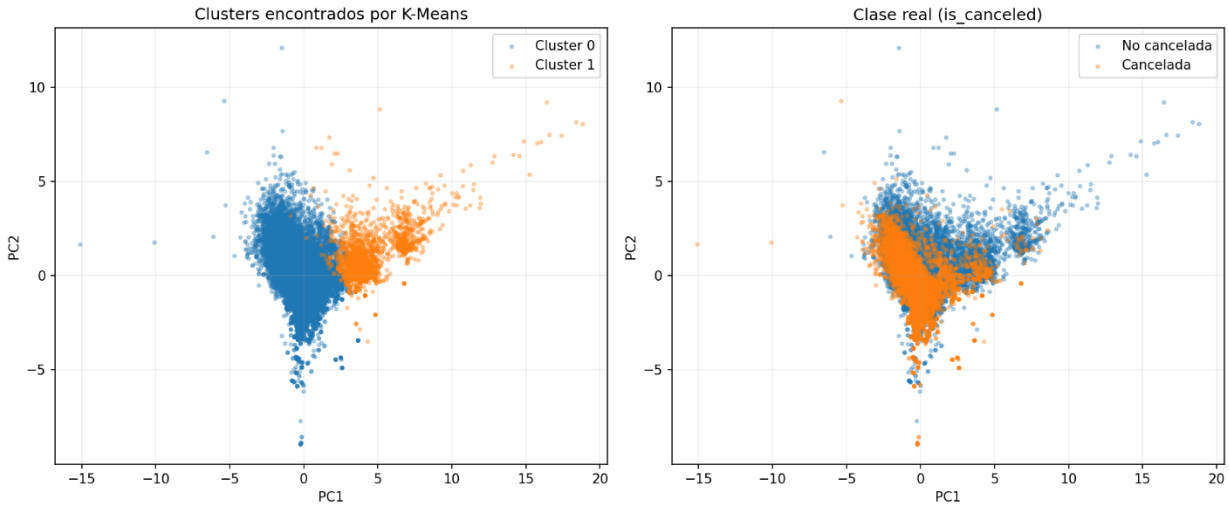


Figura 46. Visualización PCA de los clusters obtenidos mediante K-Means.

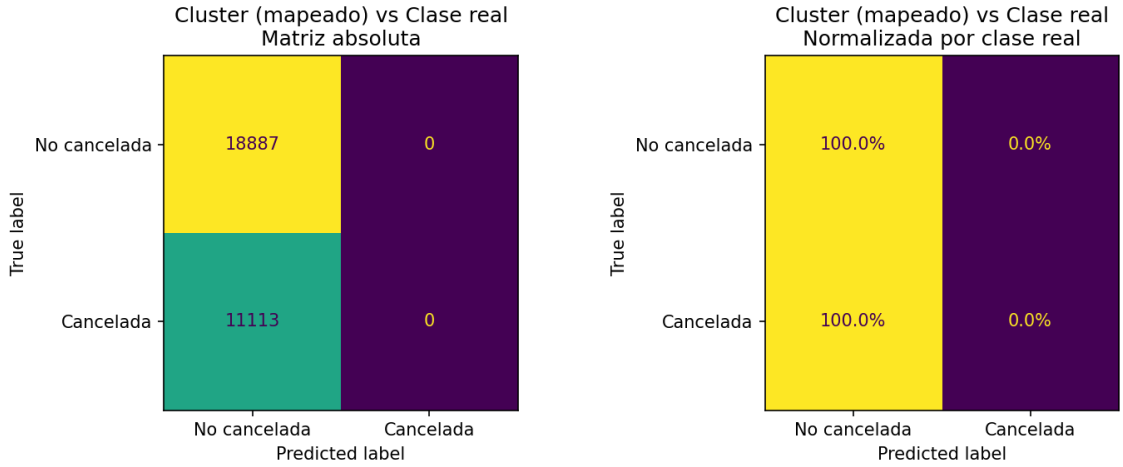


Figura 47. Matriz de comparación entre los grupos de K-Means y las clases reales.

Para comparar los agrupamientos con la variable objetivo, cada cluster se asignó a la clase real mayoritaria entre sus integrantes. Ambos grupos tuvieron mayoría de reservas no canceladas. En consecuencia, la matriz de comparación registró 18.887 reservas no canceladas asignadas a esa clase y 11.113 reservas canceladas asignadas también a no canceladas, sin asignaciones a la clase cancelada. La proporción de coincidencias fue de 0.6296, equivalente a predecir siempre la clase mayoritaria en esta muestra.

Así, obtener dos grupos no permitió recuperar las dos clases de cancelación mediante esta regla de asignación. El agrupamiento identificó diferencias entre reservas, pero esas diferencias

no se tradujeron en una separación útil de la variable objetivo bajo la representación y configuración evaluadas.

CONCLUSIÓN

El desarrollo permitió aplicar y comparar diferentes algoritmos de aprendizaje automático en dos problemas con características distintas: la estimación de un valor continuo mediante regresión y la clasificación binaria de reservas hoteleras. En ambos casos se siguió una estrategia experimental común basada en separación Hold-Out 70/30, validación cruzada de 5 folds y preprocesamiento integrado en pipelines, manteniendo el conjunto TEST aislado durante las etapas de selección y ajuste. Esta organización permitió reducir el riesgo de fuga de información y obtener una evaluación sobre datos que no participaron directamente en el entrenamiento.

En el problema de regresión, los resultados mostraron diferencias relativamente pequeñas entre los modelos finales. Random Forest obtuvo el menor RMSE y el mayor R^2 en TEST, aunque KNN y el Árbol de Decisión podado presentaron valores cercanos. El análisis de los atributos mediante árboles de profundidad uno permitió identificar a Sleep_Hours, PlayTime_Hours y Age como variables con mayor capacidad predictiva individual. Sin embargo, estos resultados no implican que dichas variables expliquen por sí solas el tiempo de reacción, ya que el desempeño final depende de la combinación de múltiples características y de relaciones que pueden no ser lineales.

El análisis de complejidad también mostró la importancia de controlar el crecimiento de los árboles. El árbol sin poda presentó un RMSE de 21.841 ms y un R^2 negativo en TEST, mientras que el árbol podado alcanzó un RMSE de 15.332 ms y un R^2 de 0.3502. Esto evidencia que una estructura más compleja no necesariamente produce una mejor capacidad de generalización y que la regularización cumple una función importante cuando el modelo puede adaptarse excesivamente a los datos de entrenamiento.

En clasificación, los tres modelos consiguieron discriminar entre reservas canceladas y no canceladas, aunque con diferencias en sus métricas. Random Forest alcanzó los valores más altos de Accuracy, Balanced Accuracy, F1-score y ROC-AUC entre las configuraciones evaluadas, con 0.8463, 0.8382, 0.7955 y 0.9288, respectivamente. Además, obtuvo un Recall de 0.8071, lo que indica que identificó una mayor proporción de las reservas pertenecientes a la clase de

cancelación. El Árbol de Decisión obtuvo un comportamiento cercano, mientras que KNN presentó valores inferiores en las principales métricas evaluadas.

El análisis individual de los atributos permitió observar que lead_time presentó la mayor capacidad inicial para separar las clases, seguido por variables como country, total_of_special_requests y booking_changes. Sin embargo, al igual que en regresión, el desempeño de una variable analizada de manera aislada no debe interpretarse como una explicación completa del fenómeno. La clasificación final depende de la combinación de atributos y de las relaciones que cada algoritmo consigue aprender.

El análisis no supervisado mediante K-Means permitió complementar la clasificación supervisada desde otra perspectiva. Los agrupamientos identificados muestran perfiles internos diferentes dentro de las reservas, pero no coinciden necesariamente con las categorías de cancelación y no cancelación. Esto permite distinguir entre dos objetivos que no deben confundirse: K-Means busca encontrar estructuras o grupos presentes en los datos sin utilizar una variable objetivo, mientras que los modelos supervisados utilizan la información conocida de is_canceled para aprender a clasificar nuevas observaciones.

En conjunto, los resultados muestran que la evaluación de un modelo no debería depender de una sola métrica ni únicamente del desempeño obtenido durante el entrenamiento. La comparación entre validación cruzada y TEST, el análisis de complejidad, la revisión de variables y la observación de los errores permiten obtener una visión más completa del comportamiento de cada algoritmo. El trabajo también evidencia que un modelo con mayor complejidad no necesariamente generaliza mejor y que la elección del algoritmo debe considerar conjuntamente desempeño predictivo, estabilidad, interpretabilidad y costo computacional.